



第3章 栈和队列

提纲
CONTENTS

3.1 栈

3.2 队列



3.1 栈

3.1.1 栈的定义

栈是一种只能在一端进行插入或删除操作的线性表。

线性表

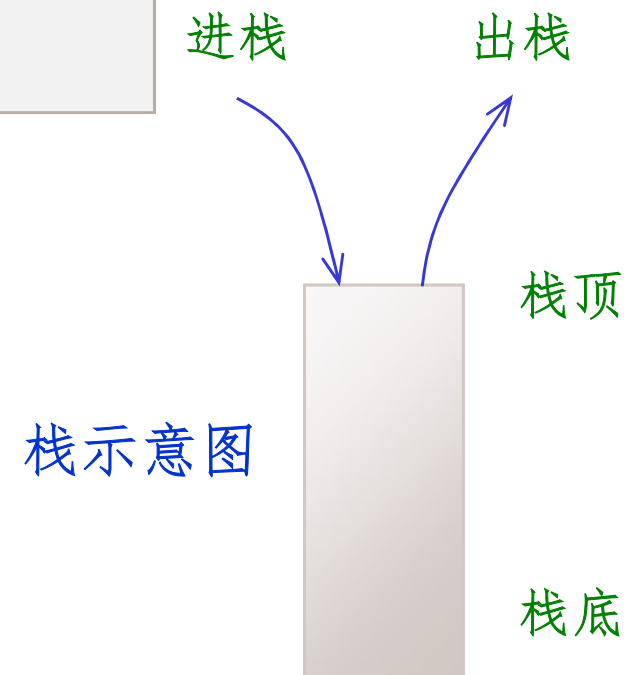


栈只能选取同一个端点进行插入和删除操作



栈的几个概念

- 允许进行插入、删除操作的一端称为栈顶。
- 表的另一端称为栈底。
- 当栈中没有数据元素时，称为空栈。
- 栈的插入操作通常称为进栈或入栈。
- 栈的删除操作通常称为退栈或出栈。



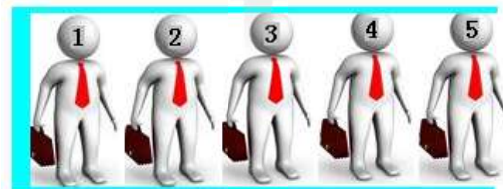
栈的主要特点是“**后进先出**”，即后进栈的元素先出栈。栈也称为**后进先出表**。

例如：



假设死胡同的宽度
恰好只够正一个人

走进死胡同的**5**人
要按相反次序退出



死胡同就是一个栈！



设一个栈的输入序列为 a, b, c, d ，则借助一个栈所得到的输出序列不可能是（ ）。

A. c, d, b, a

B. d, c, b, a

C. a, c, d, b

D. d, a, b, c

选项D是不可能的？

$d \quad c \quad b \quad a$

栈



下一步不可能出栈 a

答案为D



【例3.3】一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_1=3$ ，则 p_2 可能取值的个数是多少？

A. $n-3$

B. $n-2$

C. $n-1$

D. 无法确定

1、2、3进栈， 3出栈的结果：

n
4
...



栈



不可能是1和3
共有 $n-2$ 可能

答案为B



示例

全国考研题

一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，其出栈序列是 $p_1, p_2, p_3, \dots, p_n$ 。若 $p_2=3$ ，则 p_3 可能取值的个数是（ ）多少？

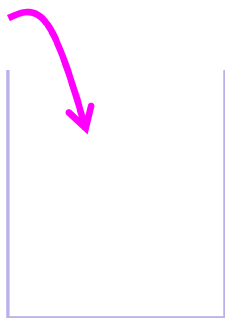
A. $n-3$

B. $n-2$

C. $n-1$

D. 无法确定

$n \quad \dots \quad 3 \quad 2 \quad 1$



不可能

解

p_3 可以取1: 1进, 2进, 2出, 3进, 3出, 1出, $\dots$ 。

p_3 可以取2: 1进, 1出, 2进, 3进, 3出, 2出, $\dots$ 。

p_3 可以取4: 1进, 1出, 2进, 3进, 3出, 4进, 4出, $\dots$ 。

p_3 可以取5: 1进, 1出, 2进, 3进, 3出, 4进, 5进, 5出, $\dots$ 。

$\dots$

p_3 可以取除了3外的任何值, 共有 $n-1$ 种可能。答案为C。



栈抽象数据类型 = 逻辑结构 + 基本运算（运算描述）

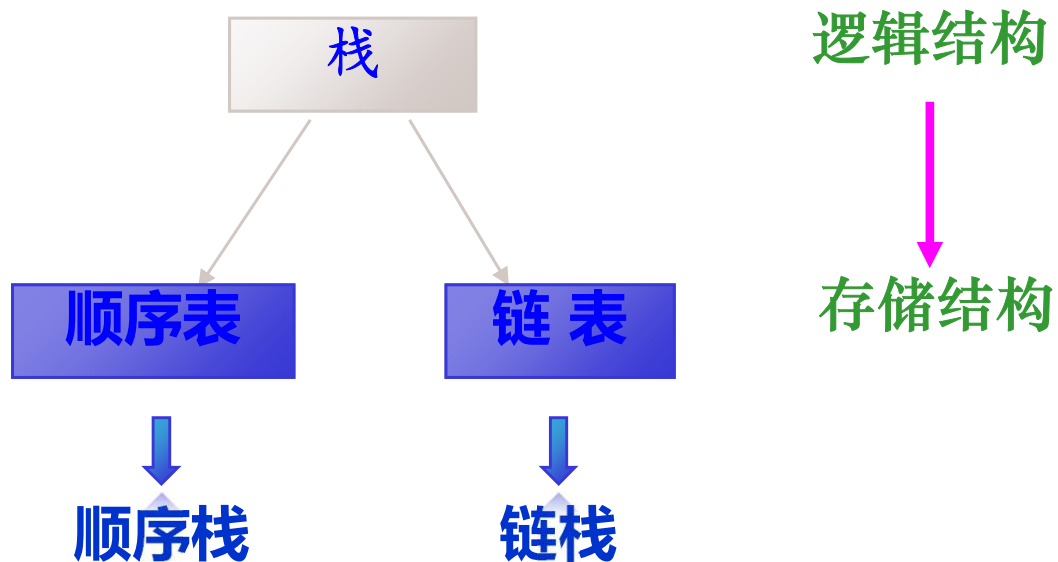
栈的几种基本运算如下：

- ① **InitStack(&s)**: 初始化栈。构造一个空栈s。
- ② **DestroyStack(&s)**: 销毁栈。释放栈s占用的存储空间。
- ③ **StackEmpty(s)**: 判断栈是否为空: 若栈s为空, 则返回真; 否则返回假。
- ④ **Push(&S, e)**: 进栈。将元素e插入到栈s中作为栈顶元素。
- ⑤ **Pop(&s, &e)**: 出栈。从栈s中退出栈顶元素, 并将其值赋给e。
- ⑥ **GetTop(s, &e)**: 取栈顶元素。返回当前的栈顶元素, 并将其值赋给e。



3.1.2 栈的顺序存储结构及其基本运算实现

栈中元素逻辑关系与线性表的相同，栈可以采用与线性表相同的存储结构。



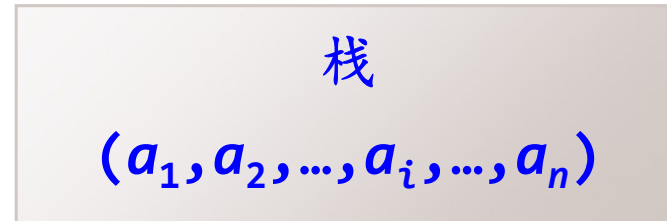


假设栈的元素个数最大不超过正整数MaxSize，所有的元素都具有同一数据类型ElemType，则可用下列方式来声明**顺序栈**类型SqStack：

```
typedef struct
{
    ElemType data[MaxSize];
    int top;           //栈顶指针
} SqStack;
```

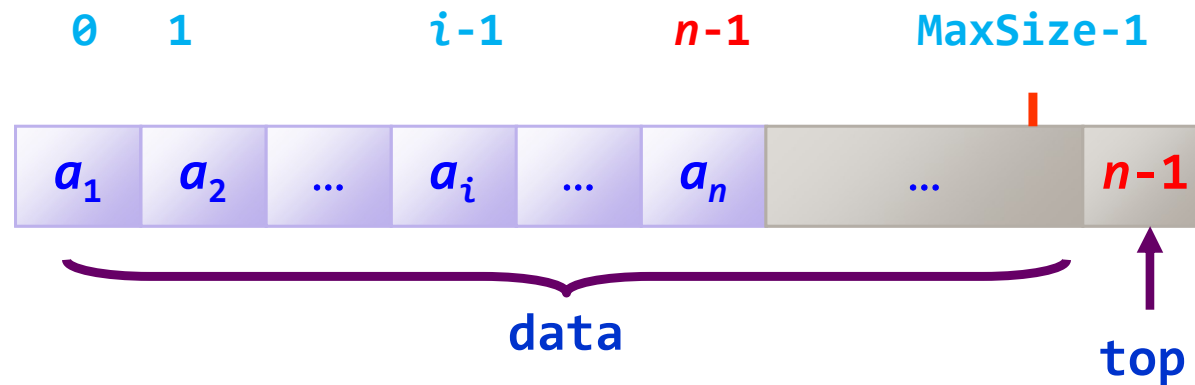


逻辑结构



直接映射

存储结构

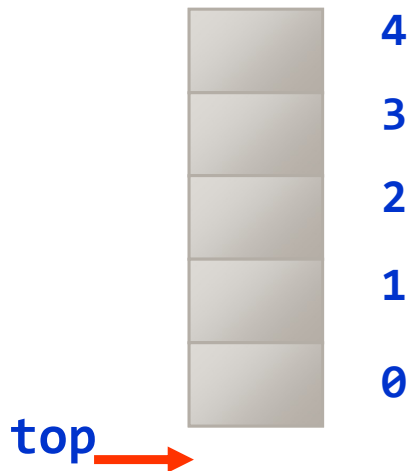


顺序栈的示意图

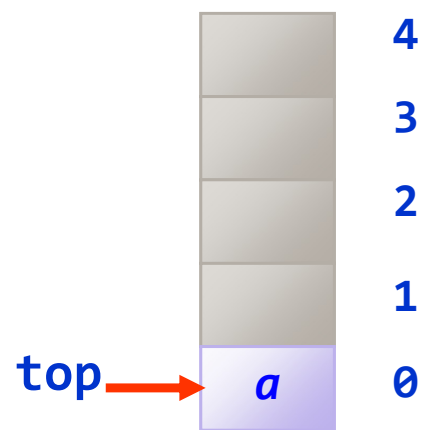


例如: $\text{MaxSize}=5$

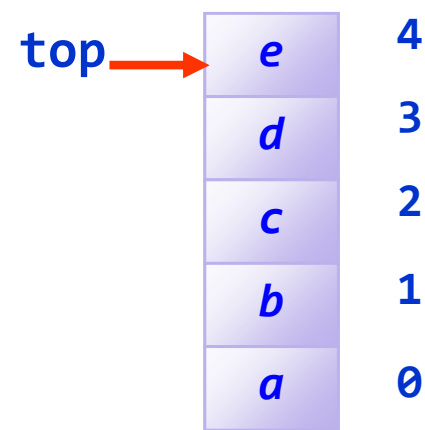
(a) 空栈



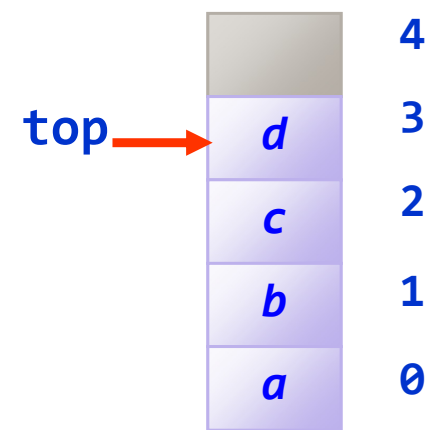
(b) a 进栈



(c) b 、 c 、 d 、 e 进栈



(d) 出栈一次

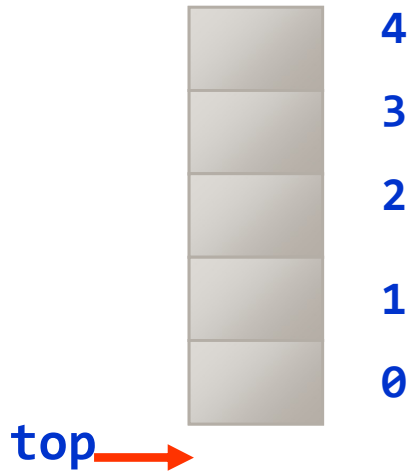


总结:

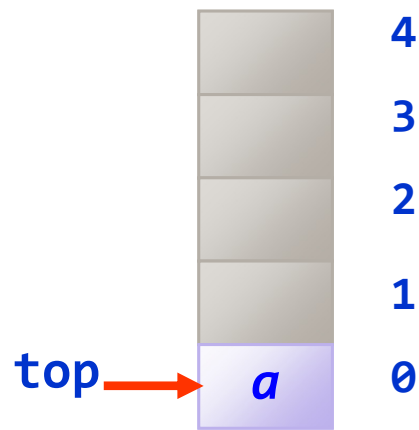
- 约定 top 总是指向栈顶元素, 初始值为 -1
- 当 $\text{top}=\text{MaxSize}-1$ 时不能再进栈 — 栈满
- 进栈时 top 增 1 , 出栈时 top 减 1



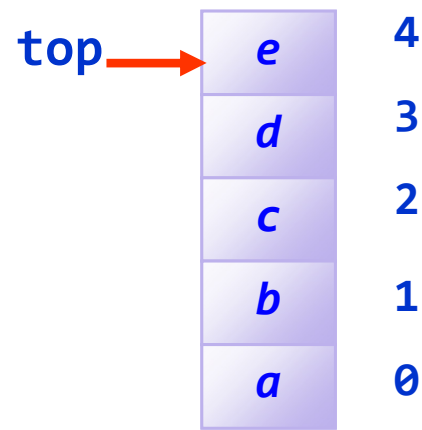
顺序栈的各种状态



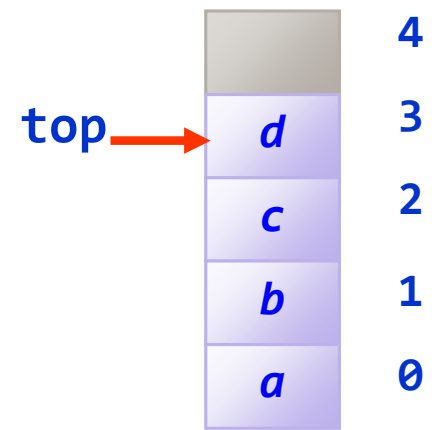
(a) 空栈



(b) a 进栈



(c) b 、 c 、 d 、 e 进栈



(d) 出栈一次

顺序栈4要素:

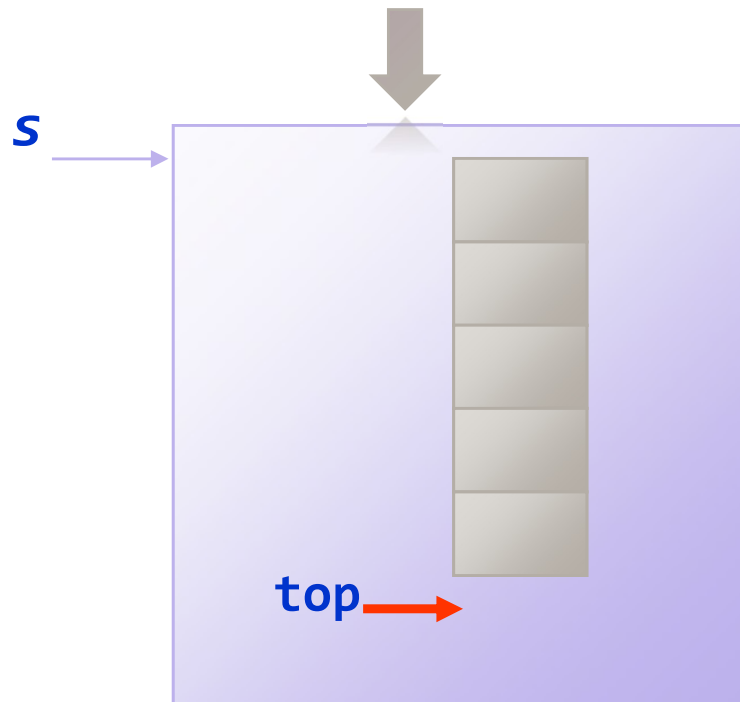
- 栈空条件: $\text{top} = -1$
- 栈满条件: $\text{top} = \text{MaxSize} - 1$
- 进栈 e 操作: $\text{top}++$; 将 e 放在 top 处
- 退栈操作: 从 top 处取出元素 e ; $\text{top}--$;



(1) 初始化栈InitStack(&s)

建立一个新的空栈s，实际上是将栈顶指针指向-1即可。

```
void InitStack(SqStack *&s)
{
    s=(SqStack *)malloc(sizeof(SqStack));
    s->top=-1;
}
```



注意：s为栈指针，top为s所指栈的栈顶指针



(2) 销毁栈DestroyStack(&s)

释放栈s占用的存储空间。

```
void DestroyStack(SqStack *&s)
{
    free(s);
}
```



(3) 判断栈是否为空 StackEmpty(s)

栈S为空的条件是 $s \rightarrow \text{top} == -1$ 。

```
bool StackEmpty(SqStack *s)
{
    return(s->top==-1);
}
```

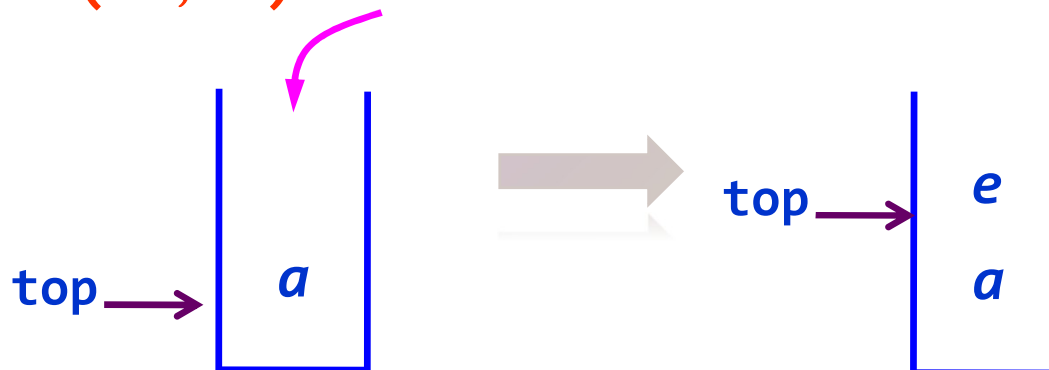


(4) 进栈Push(&s, e)

在栈不满的条件下，先将栈指针增1，然后在该位置上插入元素e。

```
bool Push(SqStack *&s, ElemType e)
{  if (s->top==MaxSize-1)    //栈满的情况，即栈上溢出
    return false;
    s->top++;                //栈顶指针增1
    s->data[s->top]=e;        //元素e放在栈顶指针处
    return true;
}
```

Push(&s, e)

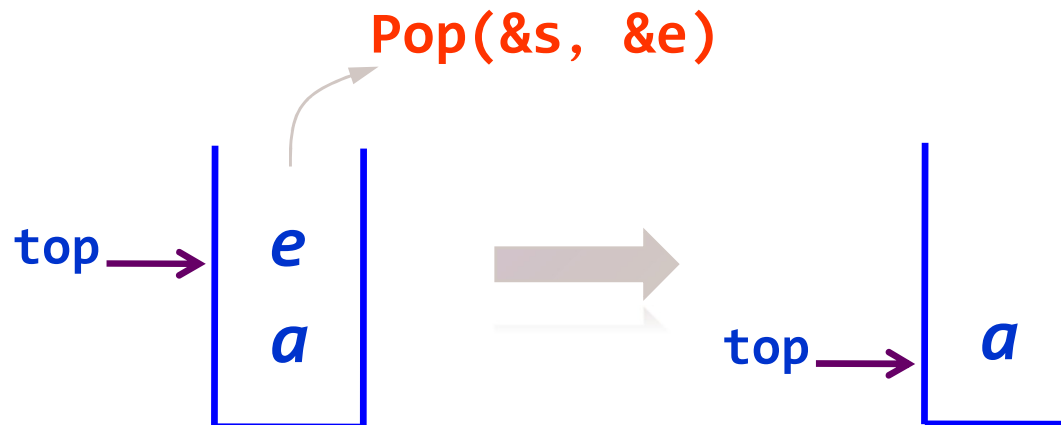




(5) 出栈Pop(&s, &e)

在栈不为空的条件下，先将栈顶元素赋给 e ，然后将栈指针减1。

```
bool Pop(SqStack *&s, ElemType &e)
{  if (s->top== -1)           //栈为空的情况，即栈下溢出
    return false;
    e=s->data[s->top];         //取栈顶指针元素的元素
    s->top--;                  //栈顶指针减1
    return true;
}
```

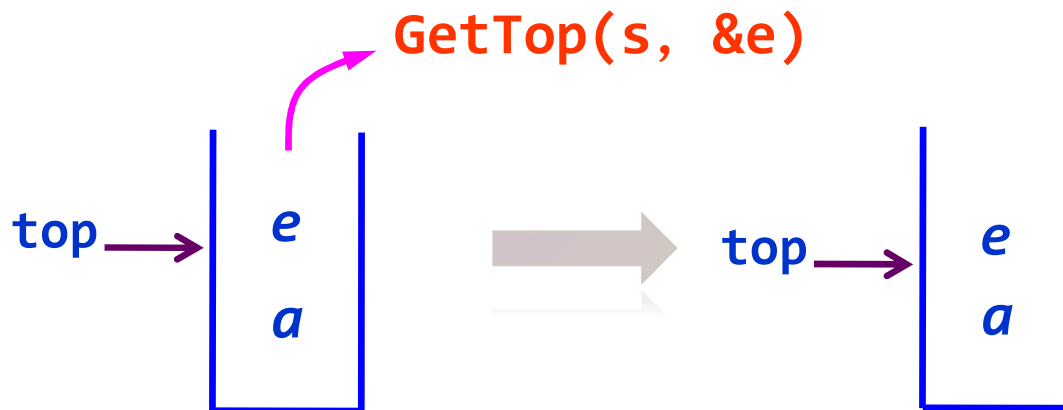




(6) 取栈顶元素GetTop(s, &e)

在栈不为空的条件下，将栈顶元素赋给 e 。

```
bool GetTop(SqStack *s, ElemType &e)
{
    if (s->top == -1)           // 栈为空的情况，即栈下溢出
        return false;
    e = s->data[s->top];        // 取栈顶指针元素的元素
    return true;
}
```





顺序栈的应用算法设计

【例3.4】 设计一个算法利用顺序栈判断一个字符串是否是对称串。
所谓**对称串**是指从左向右读和从右向左读的序列相同。

算法设计思路

- 字符串**str**的所有元素依次进栈，产生的连续出栈序列正好与**str**的顺序相反。
- 如果**str**与连续出栈序列相同，则是**对称串**。



例如, `str="abcba"`



`str`进栈并连续出栈序列`str'="abcba"`



`str=str'`, `str`是对称串



```
bool symmetry(ElemType str[])
{  int i;  ElemType e;
   SqStack *st;
   InitStack(st);                                //初始化栈

   for (i=0;str[i]!='\0';i++)                    //将串所有元素进栈
       Push(st, str[i]);                          //元素进栈

   for (i=0;str[i]!='\0';i++)
   {  Pop(st, e);                                //退栈元素e
      if (str[i]!=e)                             //若e与当前串元素不同则不是对称串
      {  DestroyStack(st);                       //销毁栈
         return false;
      }
   }

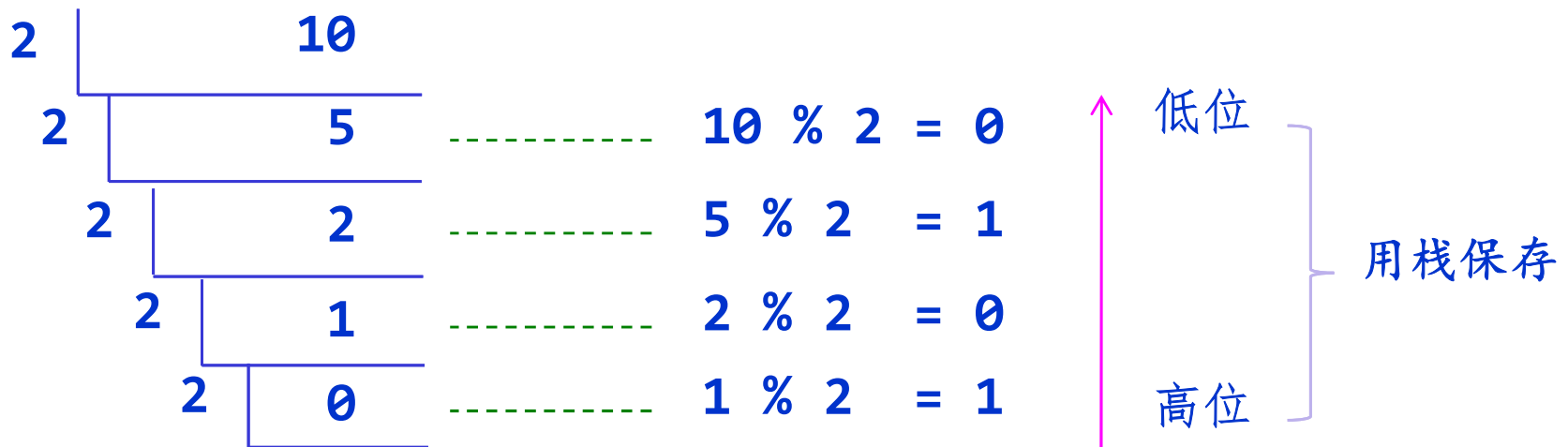
   DestroyStack(st);                             //销毁栈
   return true;
}
```



示例

设计一个算法，利用栈将一个正十进制整数转换为二进制数并输出。

十进制 $\Rightarrow$ 二进制: $n=10$





```
#include "sqstack1.cpp"
```

```
//包含整数栈的基本运算函数
```

```
void trans(int n)
```

```
{
```

```
    int e;
```

```
    SqStack *st;
```

```
    InitStack(st);
```

```
    while (n>0)
```

```
    {    Push(st,n%2);
```

```
        n=n/2;
```

```
    }
```

```
    while (!StackEmpty(st))
```

```
//输出对应的二进制数
```

```
    {    Pop(st,e);
```

```
        printf("%d",e);
```

```
    }
```

```
    printf("\n");
```

```
    DestroyStack(st);
```

```
}
```



示例

假设输入序列是 $1, 2, \dots, n$ 。设计一个算法判断通过一个栈能否得到由 $a[0..n-1]$ （为 $1, 2, \dots, n$ 的某个排列）指定出栈序列。

例如, $n=3$,

- $a[] = \{2, 3, 1\}$ 是合法出栈序列
- $a[] = \{3, 1, 2\}$ 不是合法出栈序列

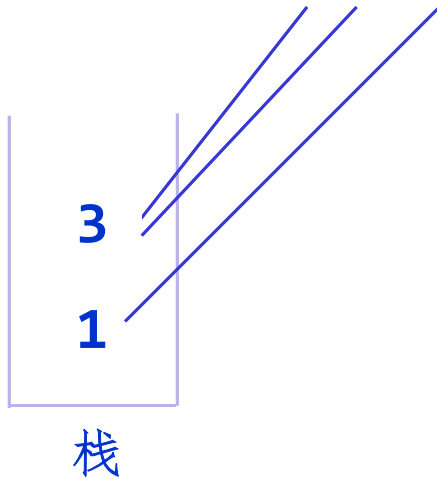


k扫描**a** (**k=0**开始), **i=1**到**n**循环:

```
{ 进栈i;  
  while(a[k]=i)  
    { 出栈i,k++; }  
}
```

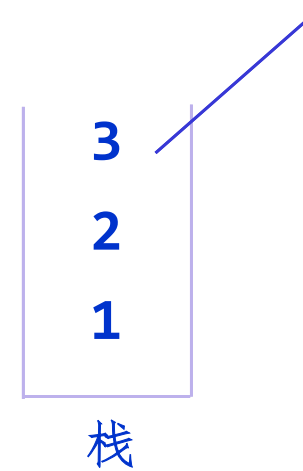
若栈空返回**true**; 否则返回**false**;

例如, $n=3$, $a[]=\{2,3,1\}$



栈空, 是出栈序列

例如, $n=3$, $a[]=\{3,1,2\}$



栈不空, 不是出栈序列



```
bool Validseq(int a[],int n)
{  int i,e,k=0;
   bool flag;
   SqStack *st;
   InitStack(st);
   for(i=1;i<=n;i++)
   {  Push(st,i);
      while (!StackEmpty(st))
      {  GetTop(st,e);
         if (a[k]==e)
         {  Pop(st,e);
            k++;
         }
         else break;
      }
   }
   flag=StackEmpty(st);
   DestroyStack(st);
   return flag;
}
```

//k扫描a的元素

//处理输入序列1、2、...、n

//i进栈

//栈不空循环

//取栈顶元素e

//匹配的情况

//不匹配时退出while循环



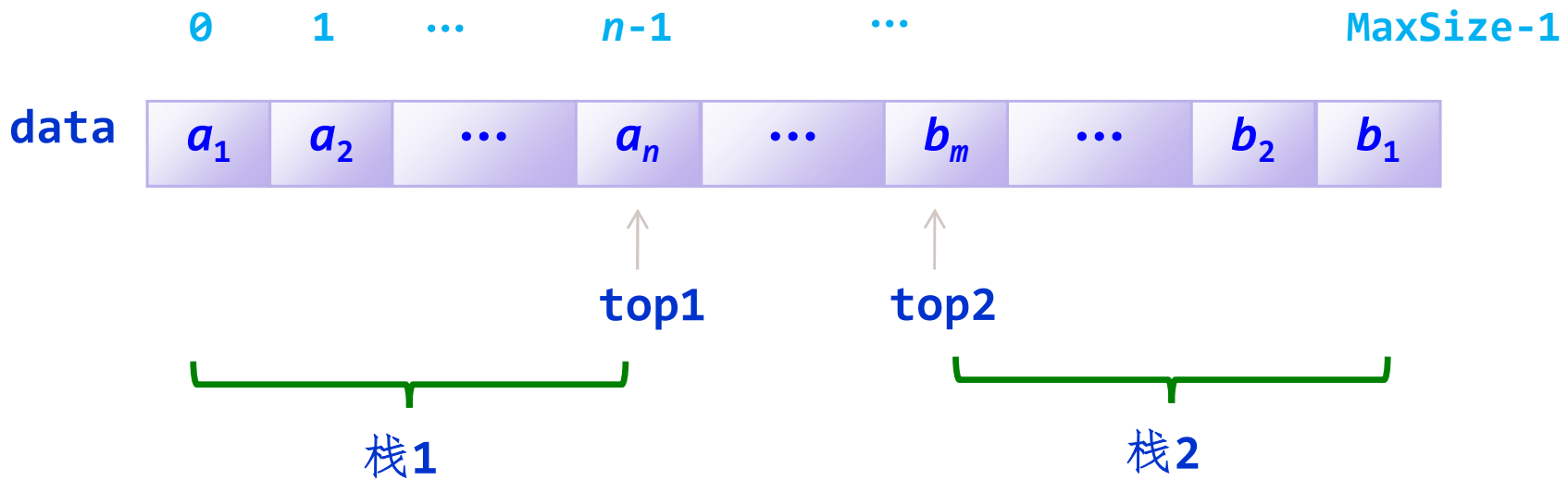
共享栈

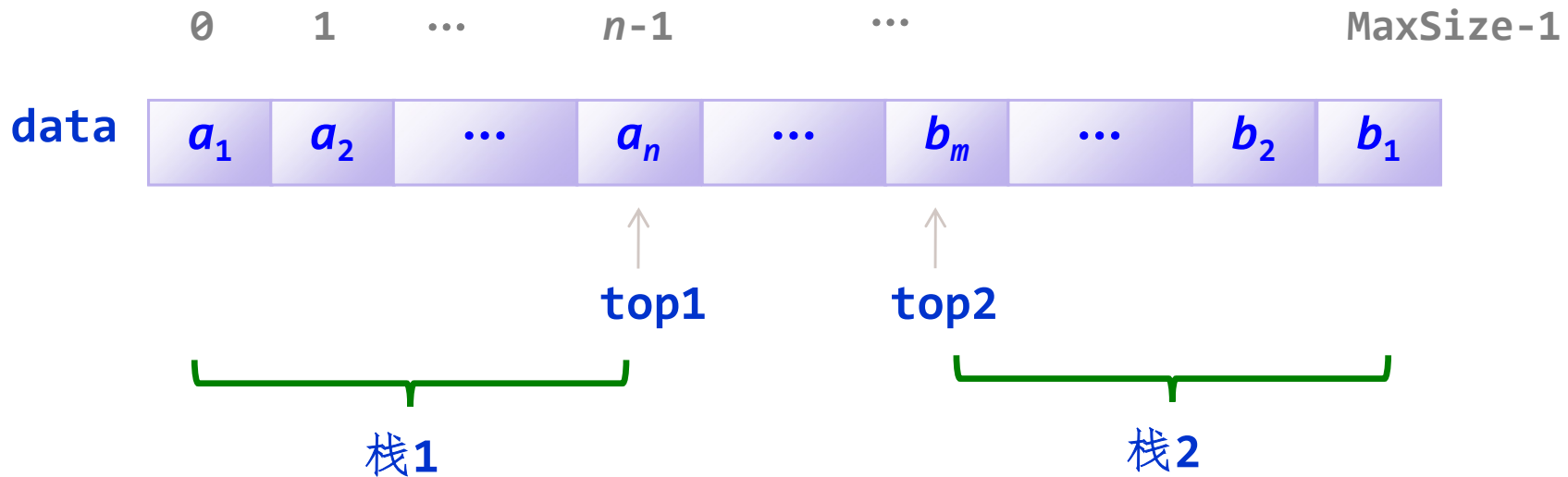
若需要用到两个相同类型的栈，可用一个数组 $\text{data}[0..\text{MaxSize}-1]$ 来实现这两个栈，这称为**共享栈**。





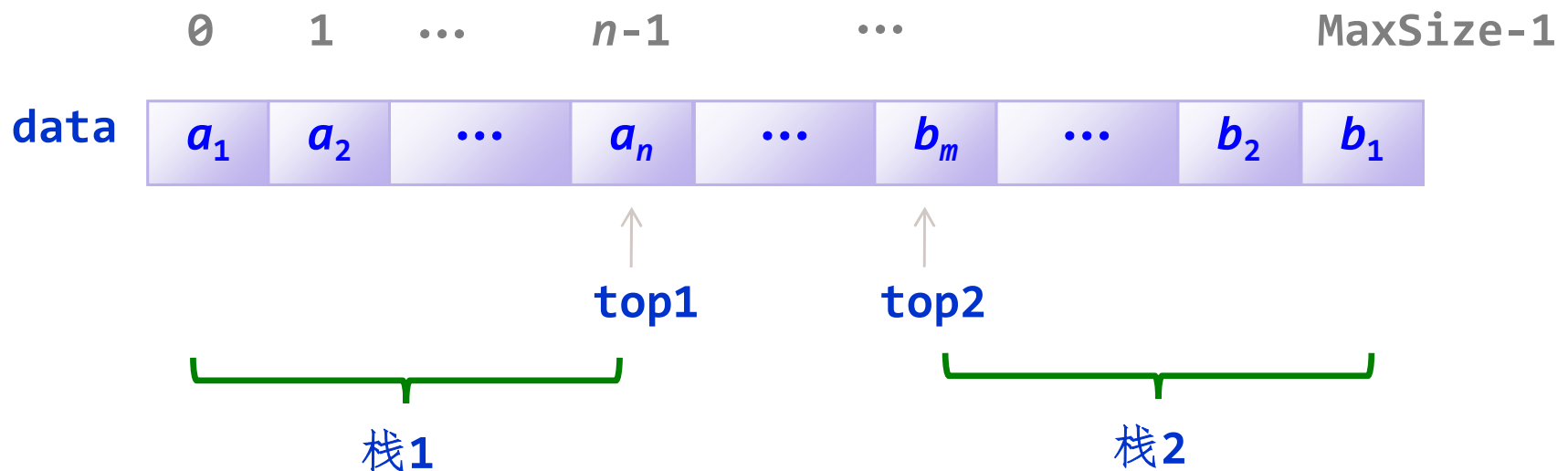
栈的一端不动，元素向另外一端伸展





共享栈类型:

```
typedef struct
{
    ElemType data[MaxSize];    //存放共享栈中元素
    int top1, top2;           //两个栈的栈顶指针
} DStack;
```



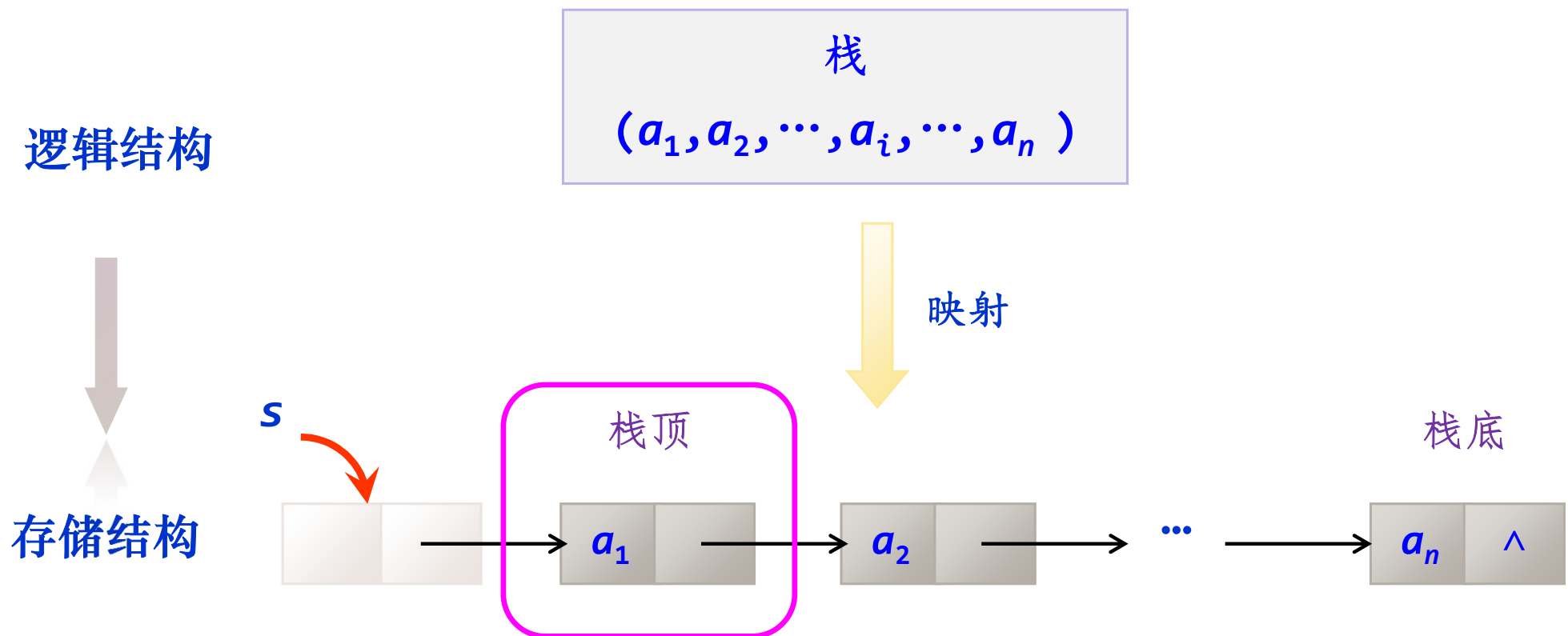
共享栈的4要素：

- **栈空条件**，栈1空： $\text{top1} == -1$ ； 栈2空： $\text{top2} == \text{MaxSize}$
- **栈满条件**： $\text{top1} == \text{top2} - 1$
- **元素x进栈操作**，进栈1操作： $\text{top1}++$; $\text{data}[\text{top1}] = x$ ； 进栈2操作： $\text{top2}--$; $\text{data}[\text{top2}] = x$ ；
- **出栈x操作**，出栈1操作， $x = \text{data}[\text{top1}]$; $\text{top1}--$ ； 出栈2操作， $x = \text{data}[\text{top2}]$; $\text{top2}++$ ；

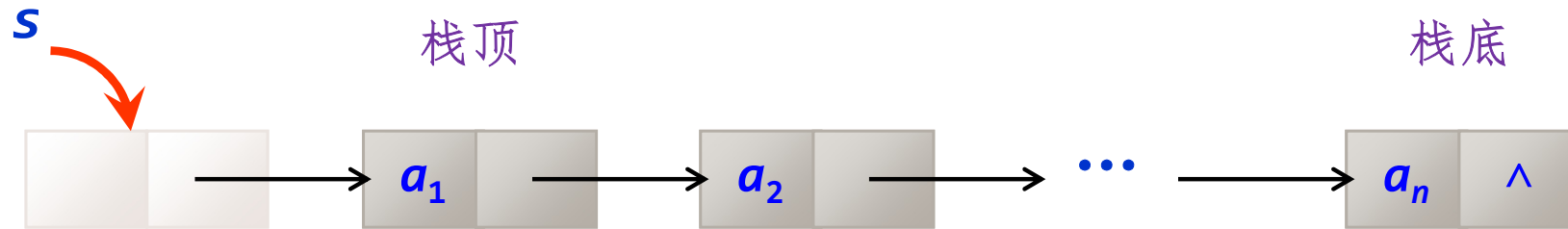


3.1.3 栈的链式存储结构及其基本运算的实现

采用链表存储的栈称为**链栈**，这里采用带头结点的单链表实现。



一个链栈的示意图



链栈的4要素：

- 栈空条件： $s \rightarrow next = NULL$
- 栈满条件： 不考虑
- 进栈 e 操作： 将存放 e 的结点插入到头结点之后
- 退栈操作： 取出头结点之后结点的元素并删除之



链栈中数据结点的类型LinkStNode声明如下：

```
typedef struct linknode
{
    ElemType data;           //数据域
    struct linknode *next;   //指针域
} LinkStNode;
```

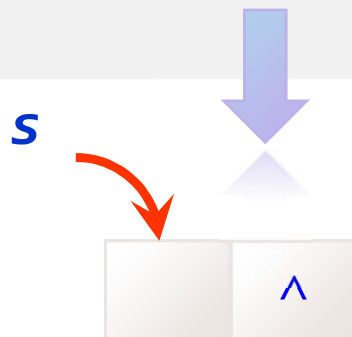


在链栈中，栈的基本运算算法如下。

(1) 初始化栈 `initStack(&s)`

建立一个空栈 `s`。实际上是创建链栈的头结点，并将其 `next` 域置为 `NULL`。

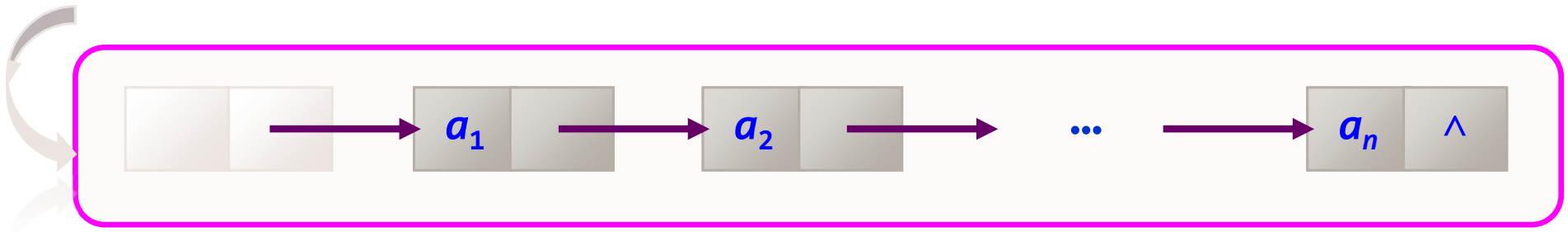
```
void InitStack(LinkStNode *&s)
{
    s=(LinkStNode *)malloc(sizeof(LinkStNode));
    s->next=NULL;
}
```





(2) 销毁栈DestroyStack(&s)

释放栈s占用的全部存储空间。



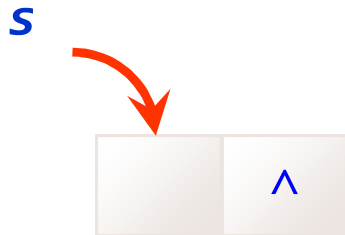
```
void DestroyStack(LinkStNode *&s)
{
    LinkStNode *p=s, *q=s->next;
    while (q!=NULL)
    {
        free(p);
        p=q;
        q=p->next;
    }
    free(p);    //此时p指向尾结点，释放其空间
}
```



(3) 判断栈是否为空StackEmpty(s)

栈S为空的条件是 $s \rightarrow next == NULL$ ，即单链表中没有数据结点。

```
bool StackEmpty(LinkStNode *s)
{
    return(s->next==NULL);
}
```



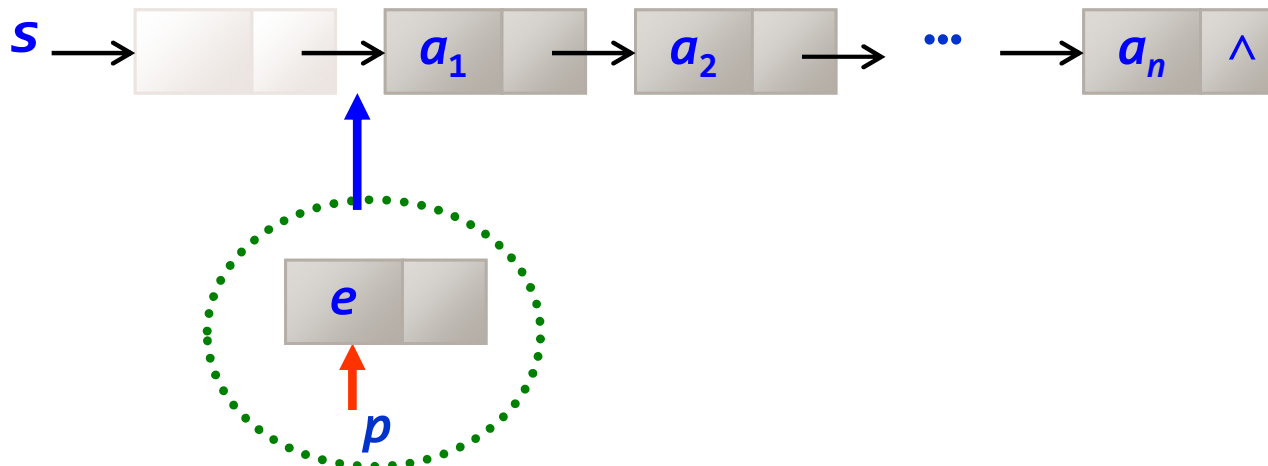
空栈的情况



(4) 进栈Push(&s, e)

将新数据结点插入到头结点之后。

```
void Push(LinkStNode *&s, ElemType e)
{
    LinkStNode *p;
    p=(LinkStNode *)malloc(sizeof(LinkStNode));
    p->data=e;           //新建元素e对应的结点p
    p->next=s->next;     //插入p结点作为开始结点
    s->next=p;
}
```

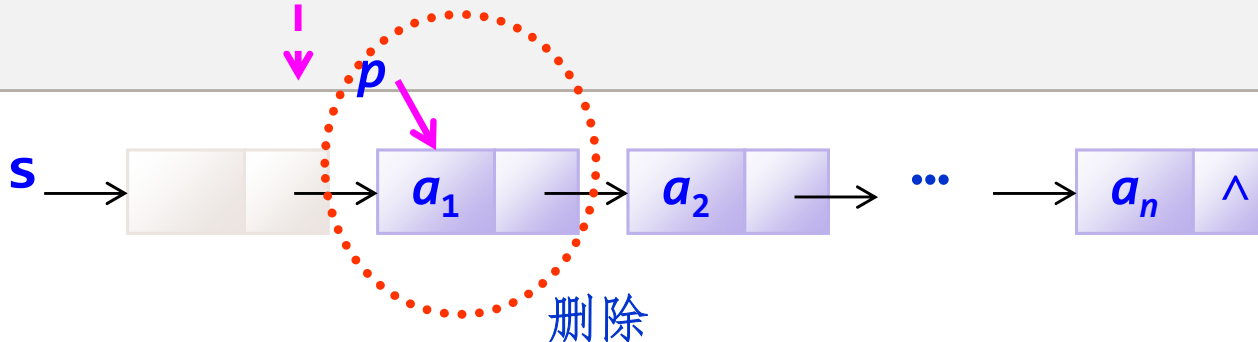




(5) 出栈Pop(&s, &e)

栈非空时，将头结点后继数据结点的数据域赋给 e ，然后将其删除。

```
bool Pop(LinkStNode *&s, ElemType &e)
{
    LinkStNode *p;
    if (s->next==NULL)           //栈空的情况
        return false;
    p=s->next;                    //p指向开始结点
    e=p->data;
    s->next=p->next;              //删除p结点
    free(p);                      //释放p结点
    return true;
}
```

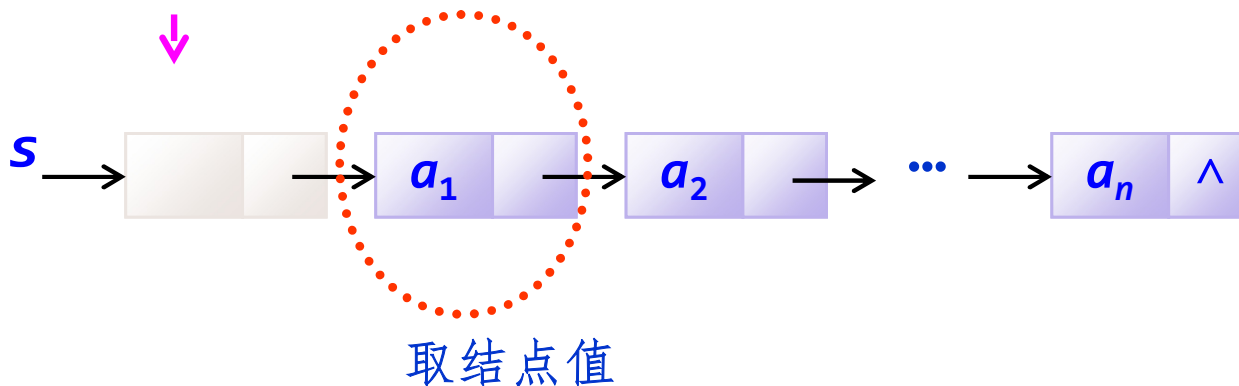




(6) 取栈顶元素GetTop(s, e)

在栈不为空的条件下，将头结点后继数据结点的数据域赋给 e 。

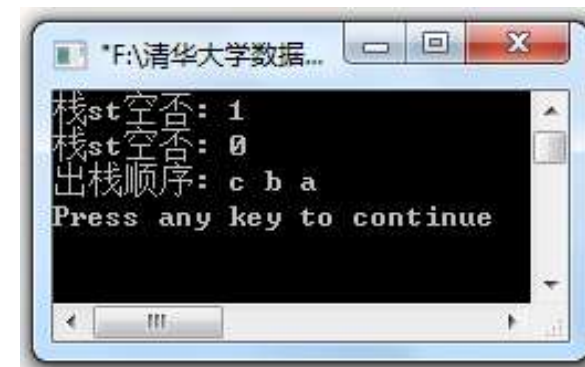
```
bool GetTop(LinkStNode *s, ElemType &e)
{
    if (s->next==NULL)           //栈空的情况
    {
        return false;
        e=s->next->data;
        return true;
    }
}
```





将字符栈类型声明及其基本运算函数存放在sqstack.c文件中

```
#include "sqstack.cpp" //包含栈的基本运算函数
void main()
{   ElemType e;
    SqStack *st;
    InitStack(st);
    printf("栈st空否: %d\n", StackEmpty(st));
    Push(st, 'a');
    Push(st, 'b');
    Push(st, 'c');
    printf("栈st空否: %d\n", StackEmpty(st));
    printf("出栈顺序: ");
    while (!StackEmpty(st))
    {   Pop(st, e);
        printf("%c ", e);
    }
    printf("\n");
    DestroyStack(st);
}
```





链栈的应用算法设计

【例3.5】编写一个算法判断输入的表达式中括号是否配对（假设只含有左、右圆括号）。

算法设计思路

一个表达式中的左右括号是按**最近位置配对的**。所以利用一个栈来进行求解。这里采用链栈。



表达式括号不配对情况的演示

例如: exp= “(()) ”

↑ ↑ ↑ ↑ ↑

(
(

① ‘(‘进栈

② ‘(‘进栈

③ 遇到’)’, 栈顶为’ (‘, 退栈

④ 遇到’)’, 栈顶为’ (‘, 退栈

⑤ 遇到’)’, 栈为空, 返回false



表达式括号配对情况的演示

例如: $\text{exp} = "(())"$

↑ ↑ ↑ ↑

(
(

① ‘(’进栈

② ‘(’进栈

③ 遇到’)’, 栈顶为’ (’, 退栈

④ 遇到’)’, 栈顶为’ (’, 退栈

栈空且 exp 扫描完, 返回true



```
bool Match(char exp[], int n)
```

```
{  int i=0; char e;
```

```
    bool match=true;
```

```
    LinkStNode *st;
```

```
    InitStack(st);
```

```
    while (i<n && match)
```

```
    {
```

```
        if (exp[i]=='(')
```

```
            Push(st, exp[i]);
```

```
//初始化栈
```

```
//扫描exp中所有字符
```



```
else if (exp[i]=='')
{
    if (GetTop(st, e)==true)
    {
        if (e!='(')
            match=false;
        else
            Pop(st, e);
    }
    else match=false;
}

i++;
}
```

//当前字符为右括号

//栈顶元素不为 '(' 时不匹配

//将栈顶元素出栈

//无法取栈顶元素时不匹配

//继续处理其他字符



```
if (!StackEmpty(st))  
    match=false;
```

```
DestroyStack(st);           //銷毀棧  
return match;  
}
```

注意：只有在表达式扫描完毕且栈空时返回true。



3.1.4 栈的应用

栈和队列都是存放多个数据的容器。通常用于存放临时数据：

- 如果后放入的数据先处理，则使用**栈**。
- 如果先放入的数据先处理，则使用**队列**。



1. 简单表达式求值



问题描述

这里限定的**简单表达式**求值问题是：用户输入一个包含“+”、“-”、“*”、“/”、正整数和圆括号的合法算术表达式，计算该表达式的运算结果。



● 数据组织

简单表达式采用字符数组 *exp* 表示，其中只含有 “+”、“-”、“*”、“/”、正整数和圆括号。

为了方便，假设是合法的算术表达式，例如，
exp = “1+2*(4+12)”；

在设计相关算法中用到栈，这里采用 **顺序栈** 存储结构。



运算算法设计

运算符位于两个操作数中间的表达式称为**中缀表达式**。例如，
"1+2*3"就是一个中缀表达式。

中缀表达式的运算规则：“先乘除，后加减，从左到右计算，先括号内，后括号外”。

因此，中缀表达式不仅要依赖运算符优先级，而且还要处理括号。



算术表达式的另一种形式是**后缀表达式**或**逆波兰表达式**，就是在算术表达式中，运算符在操作数的后面。

如"**1+2*3**"的后缀表达式为"**1 2 3 * +**"。

后缀表达式特点：

- 没有括号，已考虑了运算符的优先级。
- 只有操作数和运算符，而且越放在前面的运算符来越优先执行。

执行。



在算术表达式中，如果运算符在操作数的前面，称为**前缀表达式**，如

"1+2*3"的前缀表达式为"+ 1 * 2 3"。

- 中缀表达式: 1 + 2 * 3
 - 后缀表达式: 1 2 3 * +
 - 前缀表达式: + 1 * 2 3
- } 运算数的相对次序相同



中缀表达式的求值过程:

- 将中缀算术表达式转换成后缀表达式。
- 对该后缀表达式求值。

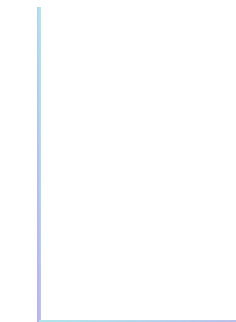


(1) 将算术表达式转换成后缀表达式

$\text{exp} \Rightarrow \text{postexp}$

扫描 exp 的所有字符:

- 数字字符直接放在 postexp 中
- 运算符通过一个栈来处理优先级



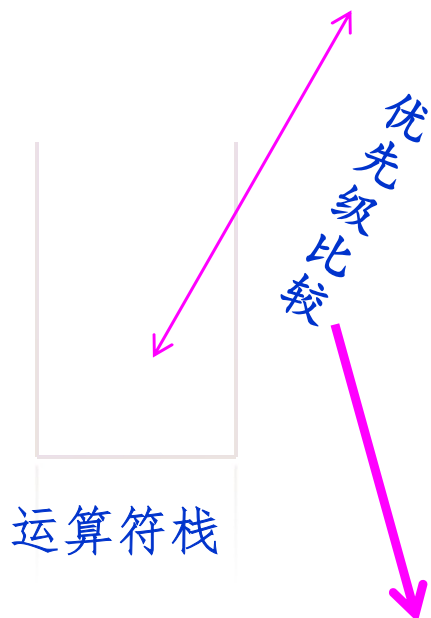
运算符栈



$\text{exp} \Rightarrow \text{postexp}$

情况1 (没有括号)

$\text{exp} = \text{"1 + 2 + 3"}$



$\text{postexp}:$

$\text{"1+2+3"} \Rightarrow \text{"1 2 + 3 +"} \quad \downarrow$

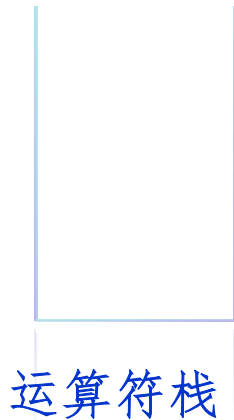
- 优先级相同：先进栈的先退栈即先执行，只有大于栈顶优先级才能直接进栈
- exp 扫描完毕，所有运算符退栈



$exp \Rightarrow postexp$

情况2 (带有括号)

$exp = "2 * (1 + 3) - 4"$



postexp:

postexp = "2 1 3 + * 4 -"

- 开始时, 任何运算符都进栈
- (: 一个子表达式开始, 进栈
- 栈顶为(: 任何运算符进栈
-): 退栈到 (
- 只有大于栈顶的优先级, 才进栈; 否则退栈



while (从**exp**读取字符**ch**, **ch**!='\0')

{ **ch**为数字: 将后续的所有数字均依次存放到**postexp**中,
并以字符'#'标志数值串结束;

ch为左括号'(': 将此括号进栈到**Optr**中;

ch为右括号')': 将**Optr**中出栈时遇到的第一个左括号'('以前的运算符依次出栈并存放到**postexp**中, 然后将左括号'('出栈;

ch为其他运算符:

if (栈空或者栈顶运算符为'(') 直接将**ch**进栈;

else if (**ch**的优先级高于栈顶运算符的优先级)
直接将**ch**进栈;

else

依次出栈并存入到**postexp**中, 直到栈顶运算符优先级小于**ch**的
优先级, 然后将**ch**进栈;

}

若**exp**扫描完毕, 则将**Optr**中所有运算符依次出栈并存放到**postexp**中。



针对简单表达式exp:

```
while (从exp读取字符ch, ch!='\0')  
{  
    ch为数字: 将后续的所有数字均依次存放到postexp中,  
               并以字符'#'标志数值串结束;  
    ch为左括号'(': 将此括号进栈到Optr中;  
    ch为右括号')': 将Optr中出栈时遇到的第一个左括号'('以前的运算符  
                   依次出栈并存放到postexp中, 然后将左括号'('出栈;  
    ch为 '+' 或 '-': 出栈运算符并存放到postexp中, 直到栈空或者栈顶为 '(',  
                   然后将 '+' 或 '-' 进栈;  
    ch为 '*' 或 '/': 出栈运算符并存放到postexp中, 直到栈空或者栈顶  
                   为 '(', '+' 或 '-', 然后将 '+' 或 '-' 进栈;  
}
```

若exp扫描完毕, 则将Optr中所有运算符依次出栈并存放到postexp中。



算法：将算术表达式`exp`转换成后缀表达式`postexp`。

```
void trans(char *exp, char postexp[])
{
    char e;
    SqStack *Optr;                //定义运算符栈指针
    InitStack(Optr);              //初始化运算符栈
    int i=0;                      //i作为postexp的下标
    while (*exp!='\0')            //exp表达式未扫描完时循环
    {
        switch(*exp)
        {
            case '(':              //判定为左括号
                Push(Optr, '(');    //左括号进栈
                exp++;              //继续扫描其他字符
                break;
        }
    }
}
```



```
case ')':  
    Pop(OpPtr, e);  
    while (e!='(')  
    {   postexp[i++] = e;  
        Pop(OpPtr, e);  
    }  
    exp++;  
    break;
```

//判定为右括号
//出栈元素e
//不为 '(' 时循环
//将e存放到postexp中
//继续出栈元素e

//继续扫描其他字符



```
case '+':  
case '-':  
    while (!StackEmpty(Optr))  
    {  
        GetTop(Optr, e);  
        if (e!='(')  
        {  
            postexp[i++] = e;  
            Pop(Optr, e);  
        }  
        else  
            break;  
    }  
    Push(Optr, *exp);  
    exp++;  
    break;
```

//判定为加或减号
//栈不空循环
//取栈顶元素e
//e不是'('
//将e存放到postexp中
//出栈元素e
//e是'('时退出循环
//将 '+' 或 '-' 进栈
//继续扫描其他字符



```
case '*':                                //判定为 '*' 或 '/' 号
case '/':
    while (!StackEmpty(Optr))           //栈不空循环
    {   GetTop(Optr, e);                 //取栈顶元素e
        if (e=='*' || e=='/')
        {   postexp[i++]=e;             //将e存放到postexp中
            Pop(Optr, e);               //出栈元素e
        }
        else                            //e为非 '*' 或 '/' 时退出循环
            break;
    }
    Push(Optr, *exp);                   //将 '*' 或 '/' 进栈
    exp++;                             //继续扫描其他字符
    break;
```



default:

```
while (*exp>='0' && *exp<='9')
```

```
{ postexp[i++]=*exp;
```

```
exp++;
```

```
}
```

```
postexp[i++]='#';
```

```
}
```

```
}
```

```
while (!StackEmpty(Optr))
```

```
{ Pop(Optr, e);
```

```
postexp[i++] = e;
```

```
}
```

```
postexp[i]='\0';
```

```
DestroyStack(Optr);
```

```
}
```

//处理数字字符

//判定为数字字符

//用#标识一个数值串结束

//此时exp扫描完毕，栈不空时循环

//出栈元素e

//将e存放到postexp中

//给postexp表达式添加结束标识

//销毁栈



(2) 后缀表达式求值

`postexp` $\Rightarrow$ 值

扫描`postexp`的所有字符:

- 数字字符: 转换为数值并进栈
- 运算符: 退栈两个操作数, 计算, 将结果进栈

操作数栈



while (从`postexp`读取字符`ch`, `ch != '\0'`)

```
{  
    ch为'+': 从Opnd栈中出栈两个数值a和b, 计算c=b+a; 将c进栈;  
    ch为'-': 从Opnd栈中出栈两个数值a和b, 计算c=b-a; 将c进栈;  
    ch为'*': 从Opnd栈中出栈两个数值a和b, 计算c=b*a; 将c进栈;  
    ch为'/': 从Opnd栈中出栈两个数值a和b, 若a不零, 计算c=b/a; 将c进栈;  
    ch为数字字符: 将连续的数字串转换成数值d, 将d进栈;  
}
```

返回`Opnd`栈的栈顶操作数即后缀表达式的值;



算法：计算后缀表达式 $postexp$ 的值。

```
double compvalue(char *postexp)
{
    double d, a, b, c, e;
    SqStack1 *Opnd;
    InitStack1(Opnd);
    while (*postexp!='\0')
    {
        switch (*postexp)
        {
            case '+':
                Pop1(Opnd, a);
                Pop1(Opnd, b);
                c=b+a;
                Push1(Opnd, c);
                break;
            // 定义为 '+' 号
            // 出栈元素 a
            // 出栈元素 b
            // 计算 c
            // 将计算结果 c 进栈
        }
    }
}
```



```
case '-':  
    Pop1(Opnd, a);  
    Pop1(Opnd, b);  
    c=b-a;  
    Push1(Opnd, c);  
    break;  
case '*':  
    Pop1(Opnd, a);  
    Pop1(Opnd, b);  
    c=b*a;  
    Push1(Opnd, c);  
    break;
```

//判定为 '-' 号
//出栈元素 a
//出栈元素 b
//计算 c
//将计算结果 c 进栈

//判定为 '*' 号
//出栈元素 a
//出栈元素 b
//计算 c
//将计算结果 c 进栈



```
case '/':                                //判定为 '/' 号
    Pop1(Opnd, a);                       //出栈元素 a
    Pop1(Opnd, b);                       //出栈元素 b
    if (a!=0)
    {   c=b/a;                           //计算 c
        Push1(Opnd, c);                 //将计算结果 c 进栈
        break;
    }
    else
    {   printf("\n\t除零错误!\n");
        exit(0);                        //异常退出
    }
    break;
```



```
default:                //处理数字字符
    d=0;                //转换成对应的数值存放到d中
    while (*postexp>='0' && *postexp<='9')
    {    d=10*d+*postexp-'0';
        postexp++;
    }
    Push1(Opnd, d);      //将数值d进栈
    break;
}
postexp++;              //继续处理其他字符
}
GetTop1(Opnd, e);       //取栈顶元素e
DestroyStack1(Opnd);    //销毁栈
return e;               //返回e
}
```



● 设计求解程序

建立如下主函数调用上述算法：

```
int main()
{   char exp[]="(56-20)/(4+2)";           //可将exp改为键盘输入
    char postexp[MaxSize];
    trans(exp, postexp);
    printf("中缀表达式:%s\n", exp);
    printf("后缀表达式:%s\n", postexp);
    printf("表达式的值:%g\n", compvalue(postexp));
    return 0;
}
```



运行结果

中缀表达式: $(56-20)/(4+2)$

后缀表达式: $56\#20\#-4\#2\#+/$

表达式的值: 6



2、用栈求解迷宫问题



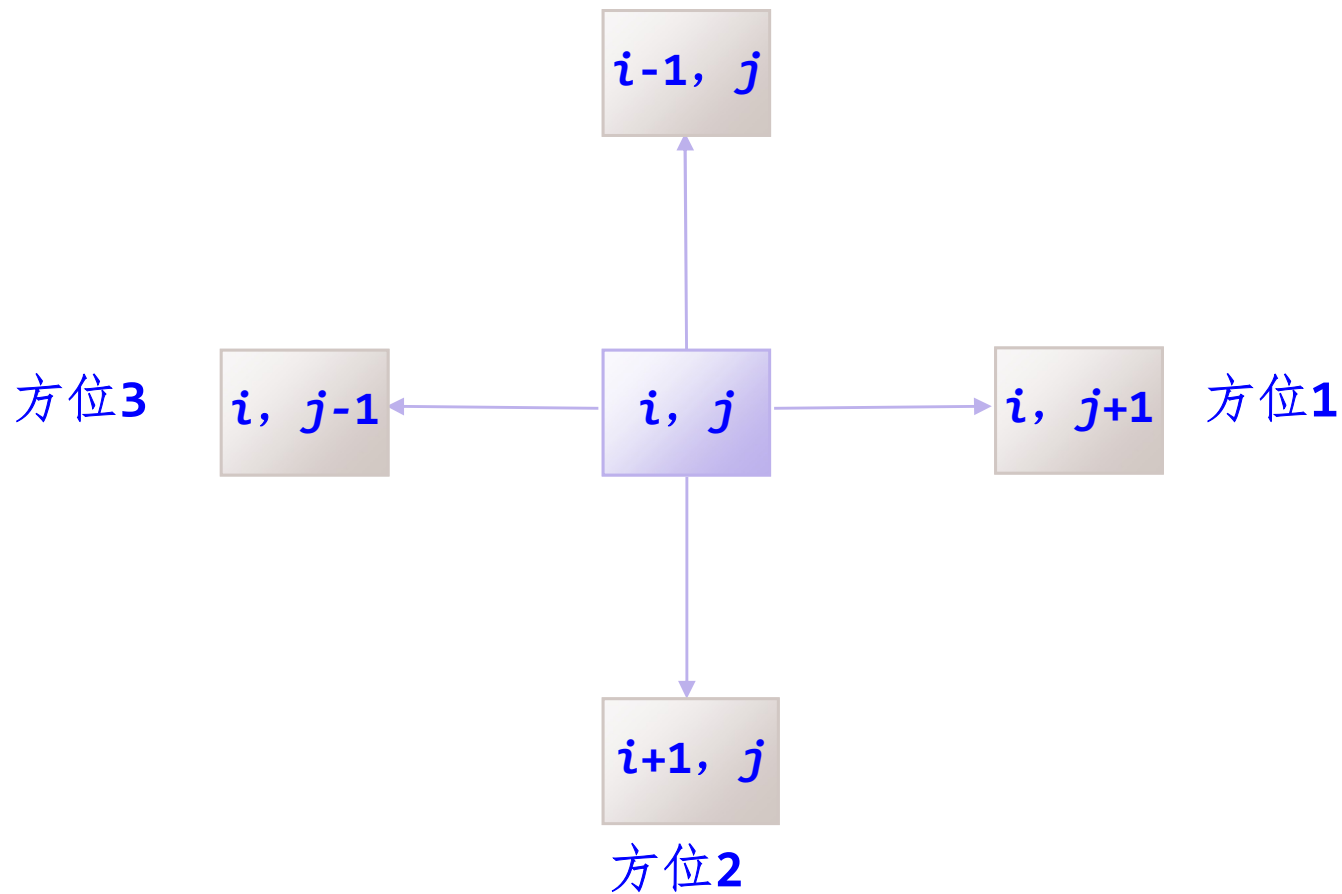
问题描述

给定一个 $M \times N$ 的迷宫图、入口与出口、行走规则。求一条从指定入口到出口的路径。

所求路径必须是简单路径，即路径不重复。



行走规则：上、下、左、右相邻方块行走。其中 (i, j) 表示一个方块
方位0

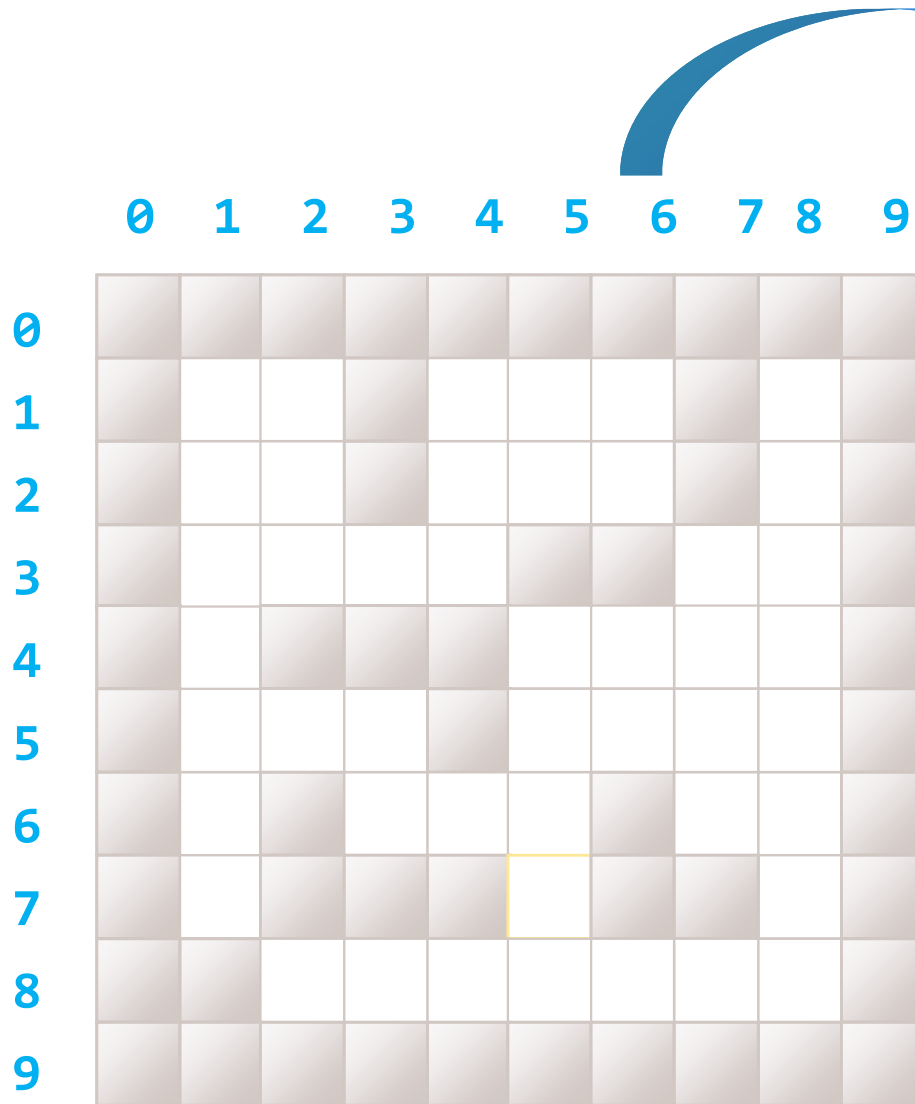






数据组织

设置一个迷宫数组mg，其中每个元素表示一个方块的状态，为0时表示对应方块是通道，为1时表示对应方块不可走。



```
int mg[M+2][N+2]=
```

```
{
```

```
{1,1,1,1,1,1,1,1,1,1},
```

```
{1,0,0,1,0,0,0,1,0,1},
```

```
{1,0,0,1,0,0,0,1,0,1},
```

```
{1,0,0,0,0,1,1,0,0,1},
```

```
{1,0,1,1,1,0,0,0,0,1},
```

```
{1,0,0,0,1,0,0,0,0,1},
```

```
{1,0,1,0,0,0,1,0,0,1},
```

```
{1,0,1,1,1,0,1,1,0,1},
```

```
{1,1,0,0,0,0,0,0,0,1},
```

```
{1,1,1,1,1,1,1,1,1,1}
```

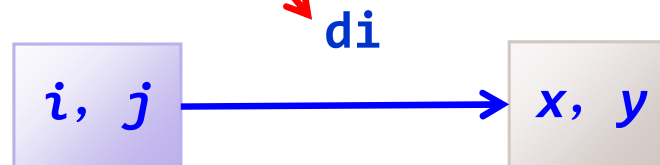
```
};
```



在算法中用到的栈采用**顺序栈**存储结构，即将栈声明为：

```
typedef struct
{
    int i;           //当前方块的行号
    int j;           //当前方块的列号
    int di;          //di是下一可走相邻方位的方位号
} Box;              //定义方块类型

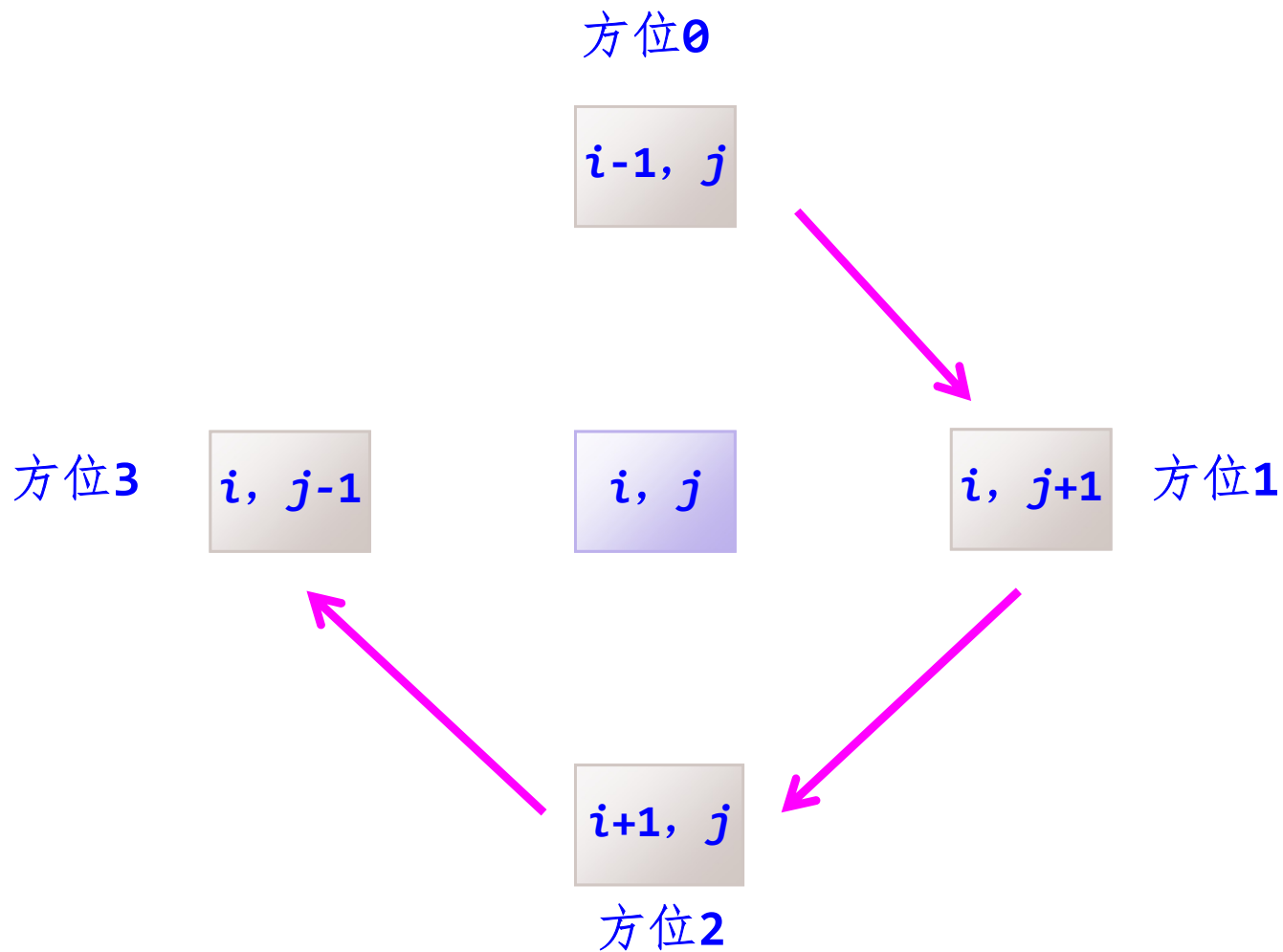
typedef struct
{
    Box data[MaxSize];
    int top;         //栈顶指针
} StType;           //声明顺序栈类型
```





算法设计

试探顺序：从方位0开始，顺时针方向





初始时，入口 (i, j) 作为当前方块。

i, j



将 $(i, j, -1)$ 进栈

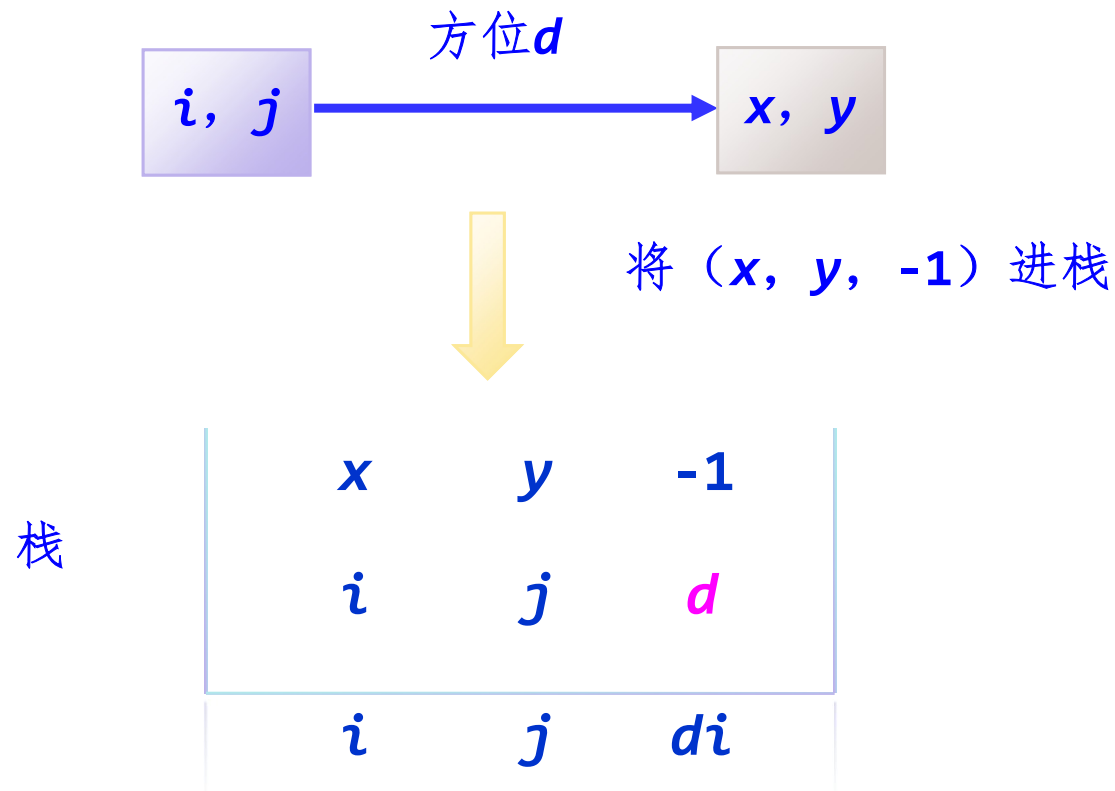
栈

i	j	-1
i	j	di

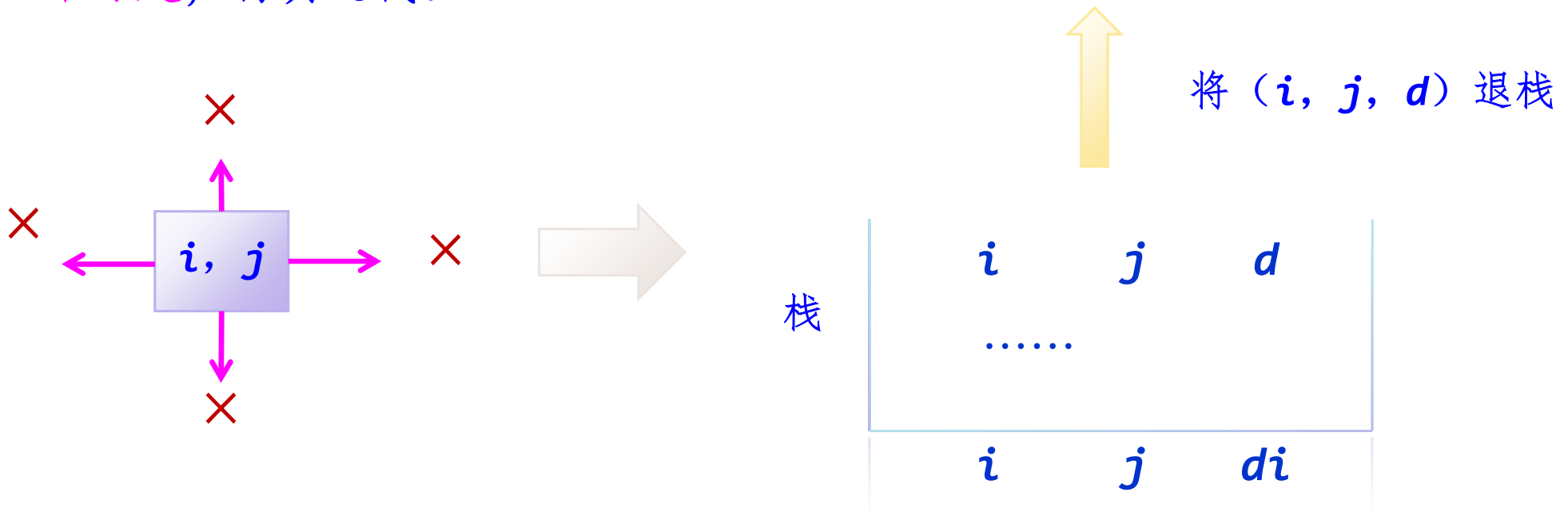
所有走过的方块都会进栈！



如果一个当前方块 (i, j) 找到一个相邻可走方块 (x, y) ，就继续从 (x, y) 走下去。

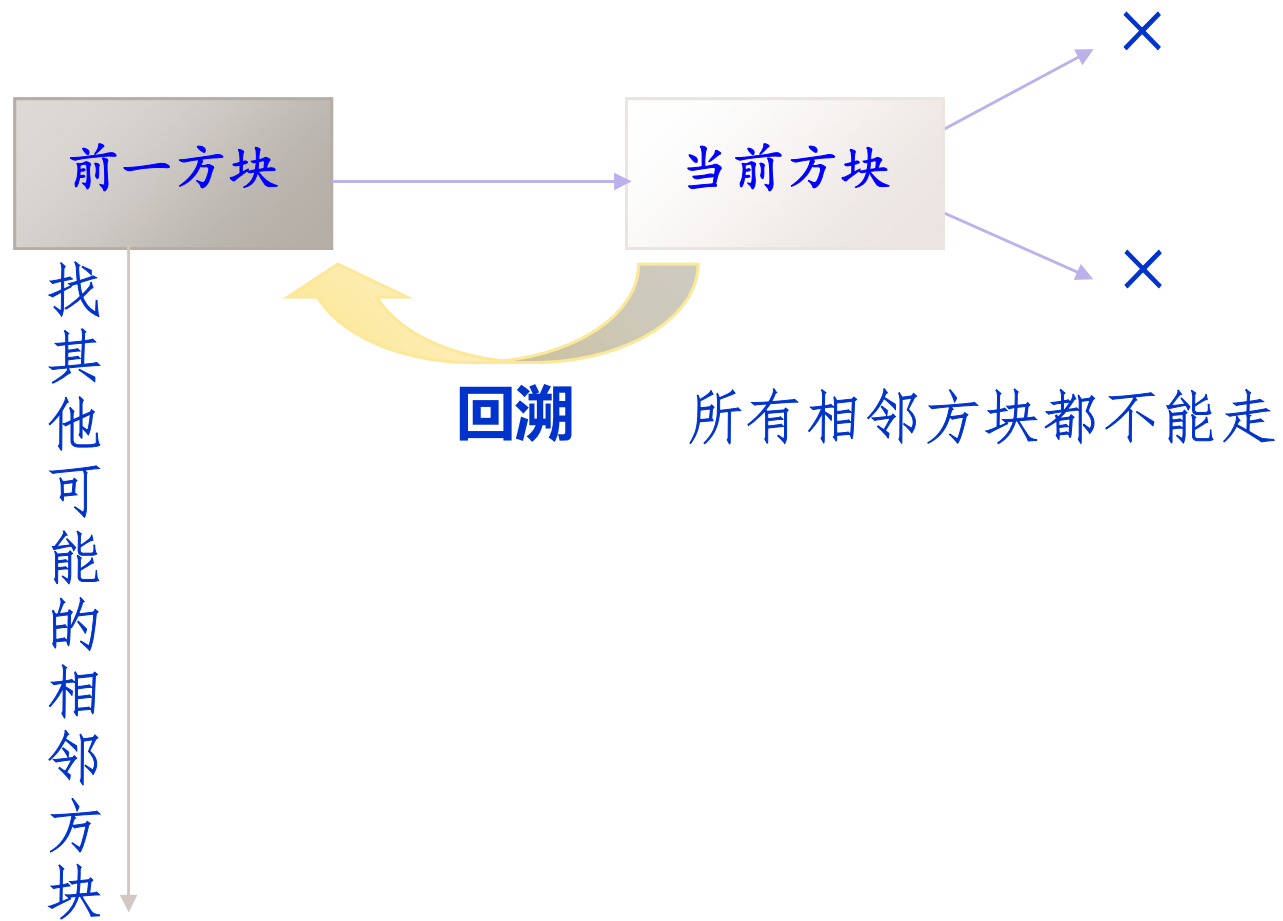


如果一个当前方块 (i, j) 没有找到任何相邻可走方块，表示此时无
路可走，将其退栈。





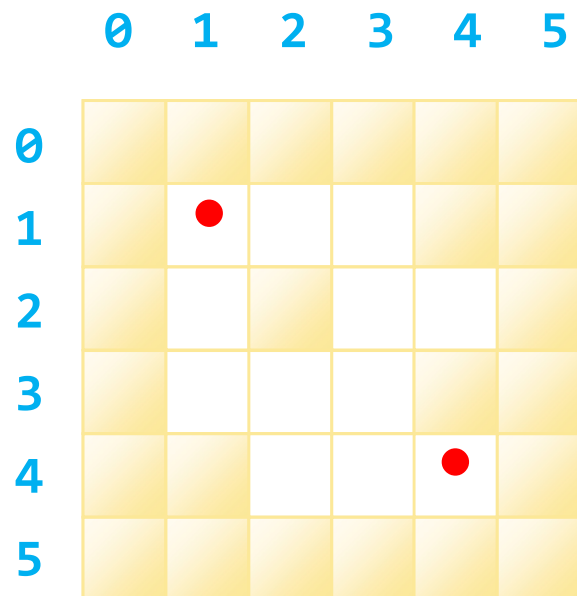
求解迷宫路径的过程：





用栈求一条迷宫路径的算法: $(xi, yi) \Rightarrow (xe, ye)$

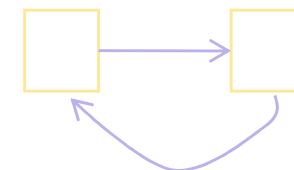
```
bool mgpath(int xi, int yi, int xe, int ye)
{
    Box path[MaxSize], e; int i, j, di, i1, j1, k; bool find;
    StType *st; //定义栈st
    InitStack(st); //初始化栈顶指针
    e.i=xi; e.j=yi; e.di=-1; //设置e为入口
    Push(st,e); //方块e进栈
    mg[xi][yi]=-1; //入口的迷宫值置为-1避免重复
}
```



1	1	-1
i	j	di

为了避免重复, 当一个方块进栈时, 将迷宫值改为-1

一个栈





```
while (!StackEmpty(st))           //栈不空时循环
{   GetTop(st,e);                 //取栈顶方块e
    i=e.i; j=e.j; di=e.di;
    if (i==xe && j==ye)           //找到了出口,输出该路径
    {   printf("一条迷宫路径如下:\n");
        k=0;
        while (!StackEmpty(st))
        {   Pop(st,e);            //出栈方块e
            path[k++]=e;          //将e添加到path数组中
        }
```



```
while (k>=1)
{
    k--;
    printf("\t(%d,%d)",path[k].i,path[k].j);
    if ((k+2)%5==0)    //每输出每5个方块后换一行
        printf("\n");
}
printf("\n");
DestroyStack(st);    //销毁栈
return true;    //输出一条迷宫路径后返回true
}
```

4	4	-1
---	---	----

4	3	1
---	---	---

3	3	1
---	---	---

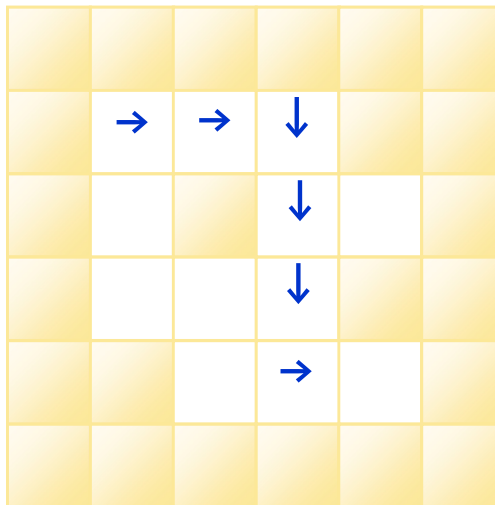
2	3	2
---	---	---

1	3	2
---	---	---

1	2	1
---	---	---

1	1	1
---	---	---

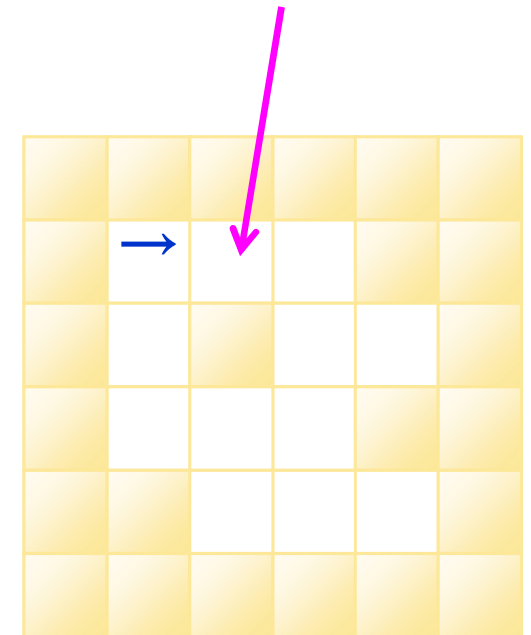
i	j	di
---	---	----





```
find=false;
while (di<4 && !find)    //找相邻可走方块(i1,j1)
{
    di++;
    switch(di)
    {
        case 0:i1=i-1; j1=j;    break;
        case 1:i1=i;   j1=j+1;  break;
        case 2:i1=i+1; j1=j;    break;
        case 3:i1=i;   j1=j-1;  break;
    }
    if (mg[i1][j1]==0) find=true;
    //找到一个相邻可走方块, 设置find为真
}
```

从入口 (1,1) 出发
找到一个可走方块
(1, 2)

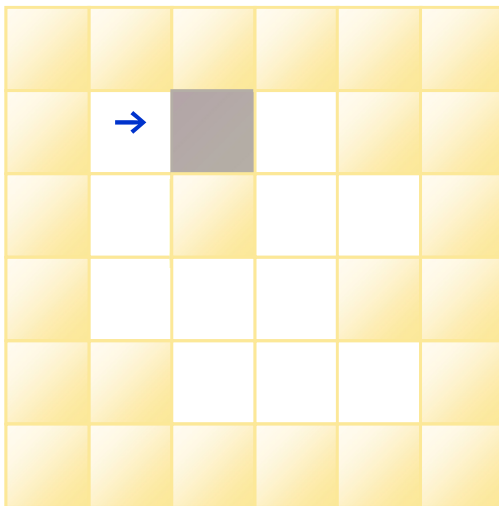




```
if (find)
{
    st->data[st->top].di=di;
    e.i=i1; e.j=j1; e.di=-1;
    Push(st,e);
    mg[i1][j1]=-1;
}
```

//找到了一个相邻可走方块(i1,j1)
//修改原栈顶元素的di值
//相邻可走方块e进栈
//(i1,j1)迷宫值置为-1避免重复

从入口 (1,1) 出发找到一个可走方块 (1, 2) :将 (1, 2, -1) 进栈



1	2	-1
1	1	1
i	j	di

一个栈



```
else
{   Pop(st,e);
    mg[e.i][e.j]=0;
}
}
DestroyStack(st);
return false;
}
```

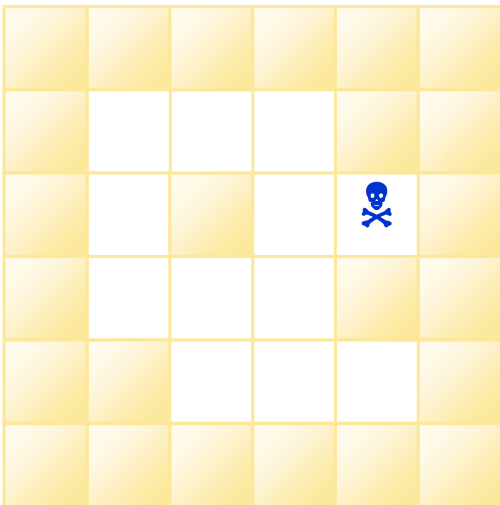
```
//没有路径可走退栈
//将栈顶方块退栈
//让退栈方块变为1

//销毁栈
//表示没有可走路径
```

疑难解答:

这里不将mg[栈顶方块]设置为0, 程序执行也是正确的, 但从原理上应该这样做, 回退后需要恢复环境!

(2,4) 方块没有通路



2 4 -1

...

i j di

退栈

一个栈



● 设计求解程序

建立如下主函数调用上述算法：

```
int main()
{  if  (!mgpath(1,1,M,N))
    printf("该迷宫问题没有解!");
    return 1;
}
```

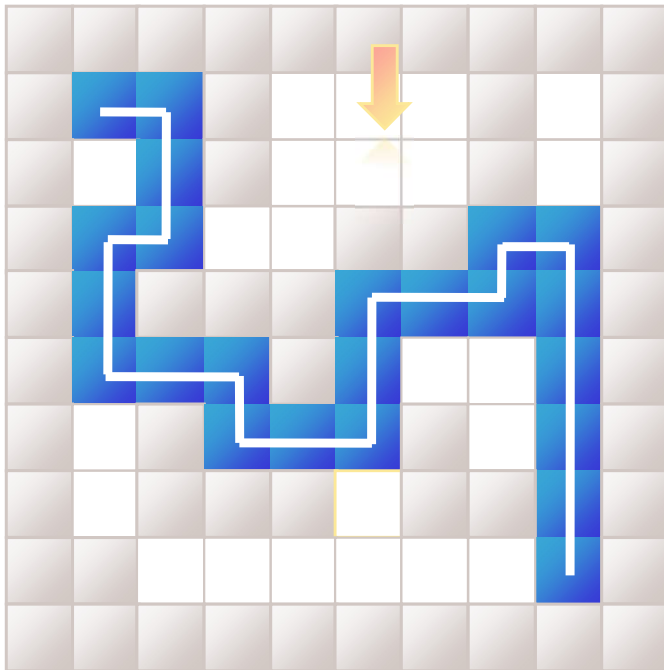


运行结果

求解结果如下：

迷宫路径如下：

(1, 1) (1, 2) (2, 2) (3, 2) (3, 1)
(4, 1) (5, 1) (5, 2) (5, 3) (6, 3)
(6, 4) (6, 5) (5, 5) (4, 5) (4, 6)
(4, 7) (3, 7) (3, 8) (4, 8) (5, 8)
(6, 8) (7, 8) (8, 8)



显然，这个解不是最优解，
即不是最短路径。为什么？



实验练习

➤ Leetcode, 题号:

- ✓ 20: 有效的括号
- ✓ 234: 回文链表
- ✓ 150: 逆波兰表达式求值
- ✓ 227: 基本计算器II



3.2 队列

3.2.1 队列的定义

队列简称**队**，它也是一种运算受限的线性表。



队列只能选取一个端点进行插入操作，另一个端点进行删除操作



队列的几个概念

- 进行插入的一端称做队尾（rear）。
- 进行删除的一端称做队首或队头（front）。
- 向队列中插入新元素称为进队或入队，新元素进队后就成为新的队尾元素。
- 从队列中删除元素称为出队或离队，元素出队后，其后继元素就成为队首元素。



队列的主要特点是**先进先出**，所以又把队列称为**先进先出表**。

例如：



假如**5**个人
过独木桥



只能按上桥的
次序过桥



这里独木桥就是一个队列



队列抽象数据类型 = 逻辑结构 + 基本运算（运算描述）

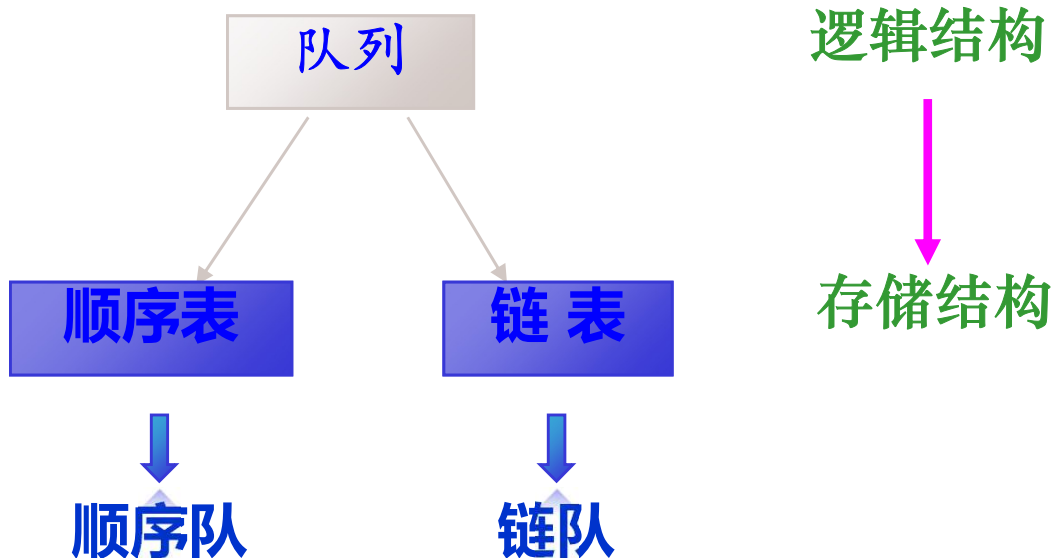
队列的基本运算如下：

- ① **InitQueue(&q)**: 初始化队列。构造一个空队列 q 。
- ② **DestroyQueue(&q)**: 销毁队列。释放队列 q 占用的存储空间。
- ③ **QueueEmpty(q)**: 判断队列是否为空。若队列 q 为空，则返回真；否则返回假。
- ④ **enqueue(&q, e)**: 进队列。将元素 e 进队作为队尾元素。
- ⑤ **deQueue(&q, &e)**: 出队列。从队列 q 中出队一个元素，并将其值赋给 e 。



3.2.2 队列的顺序存储结构及其基本运算实现

队列中元素逻辑关系与线性表的相同，队列可以采用与线性表相同的存储结构。





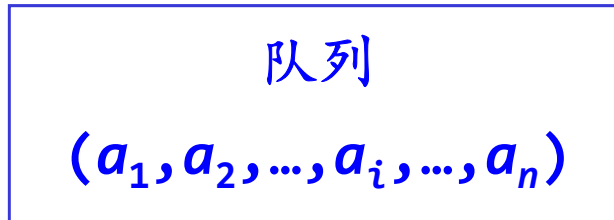
顺序队类型SqQueue声明如下：

```
typedef struct
{
    ElemType data[MaxSize];
    int front, rear;           //队首和队尾指针
} SqQueue;
```

因为队列两端都在变化，所以需要两个指针来标识队列的状态。

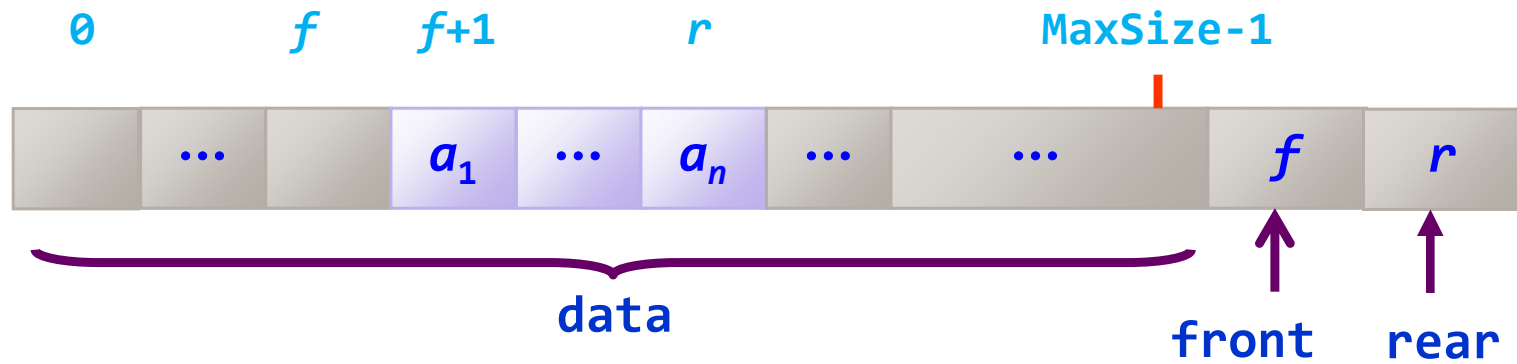


逻辑结构



直接映射

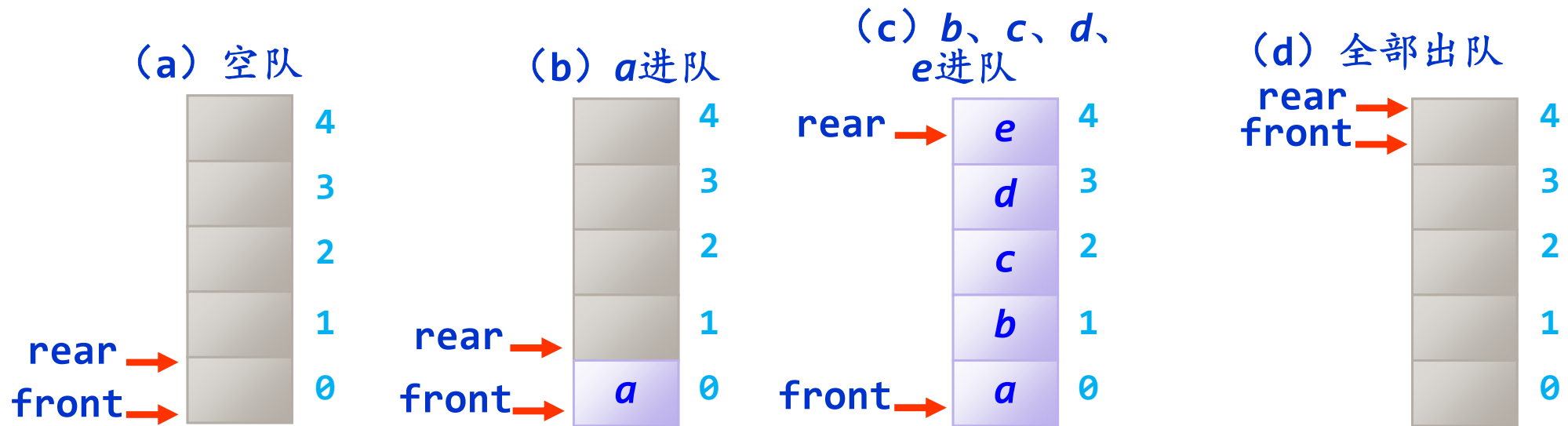
存储结构



顺序队的示意图



例如: $\text{MaxSize}=5$

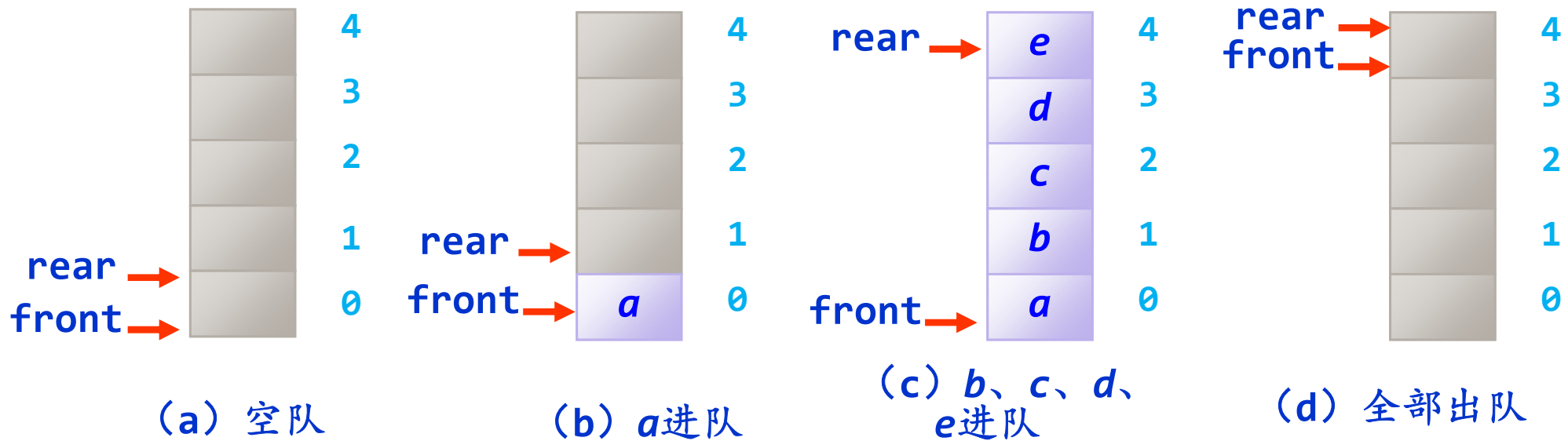


总结

- 约定 **rear** 总是指向队尾元素
- 元素进队, **rear** 增1
- 约定 **front** 指向当前队中队头元素的前一位置
- 元素出队, **front** 增1
- 当 **rear=MaxSize-1** 时不能再进队



队列的各种状态



顺序队的4要素（初始时 $\text{front}=\text{rear}=-1$ ）：

- 队空条件： $\text{front} = \text{rear}$
- 队满条件： $\text{rear} = \text{MaxSize}-1$
- 元素 e 进队： $\text{rear}++$; $\text{data}[\text{rear}]=e$;
- 元素 e 出队： $\text{front}++$; $e=\text{data}[\text{front}]$;

注意： rear 指向队尾元素； front 指向队头元素的前一个位置。

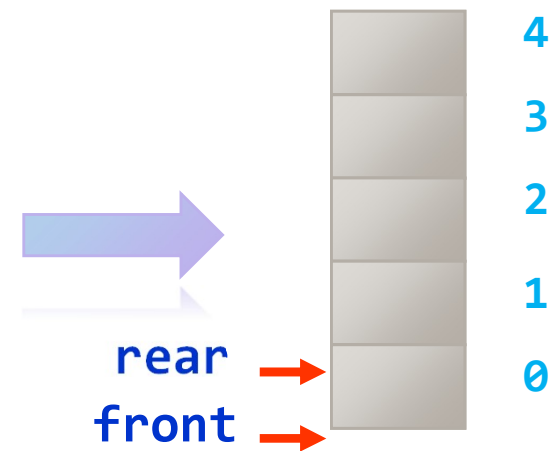


1、顺序队中实现队列的基本运算算法

(1) 初始化队列InitQueue(q)

构造一个空队列 q 。将 $front$ 和 $rear$ 指针均设置成初始状态即-1值。

```
void InitQueue(SqQueue *&q)
{
    q=(SqQueue *)malloc (sizeof(SqQueue));
    q->front=q->rear=-1;
}
```





(2) 销毁队列DestroyQueue(q)

释放队列 q 占用的存储空间。

```
void DestroyQueue(SqQueue *&q){  
    free(q);  
}
```



(3) 判断队列是否为空QueueEmpty(q)

若队列q满足 $q \rightarrow \text{front} == q \rightarrow \text{rear}$ 条件，则返回true；否则返回false。

```
bool QueueEmpty(SqQueue *q)
{
    return(q->front==q->rear);
}
```



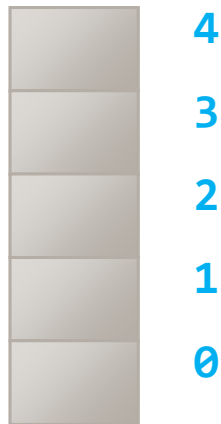
(4) 进队列enQueue(q, e)

在队列不满的条件下，先将队尾指针`rear`循环增1，然后将元素添加到该位置。

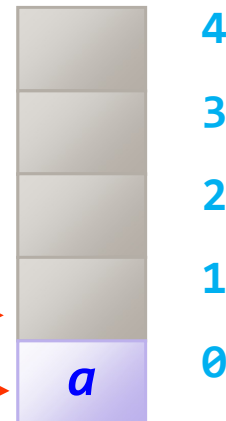
```
bool enQueue(SqQueue *&q, ElemType e)
{   if (q->rear==MaxSize-1)           //队满上溢出
        return false;
    q->rear++;
    q->data[q->rear]=e;
    return true;
}
```

空队时元素 a 进队:

rear →
front →



rear →
front →



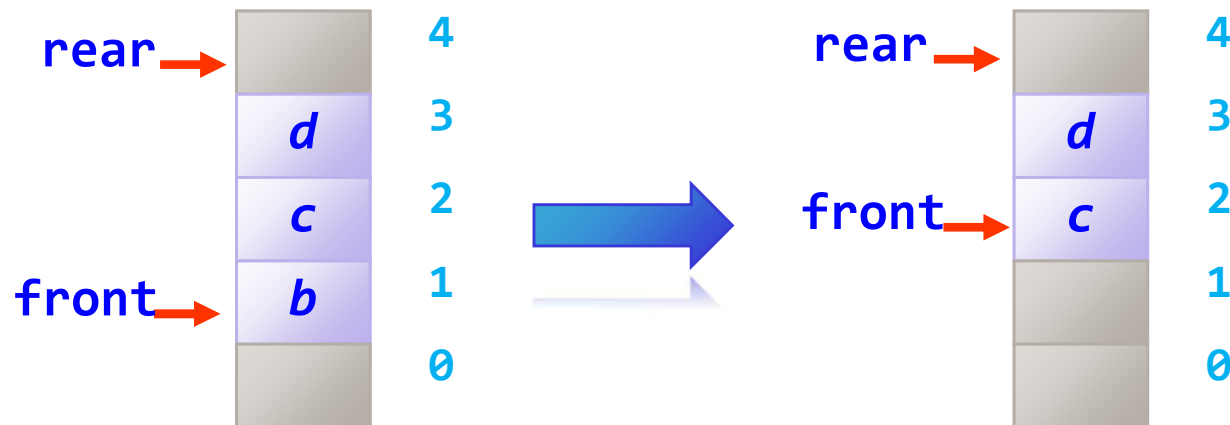


(5) 出队列deQueue(q, e)

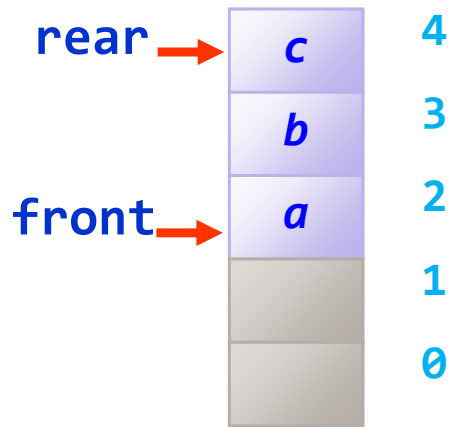
在队列 q 不为空的条件下，将队首指针 $front$ 循环增1，并将该位置的元素值赋给 e 。

```
bool deQueue(SqQueue *&q, ElemType &e)
{   if (q->front==q->rear)           //队空下溢出
    return false;
    q->front++;
    e=q->data[q->front];
    return true;
}
```

出队一个
元素：



2、环形队列（或循环队列）中实现队列的基本运算



还有两个位置，为何不能进队？

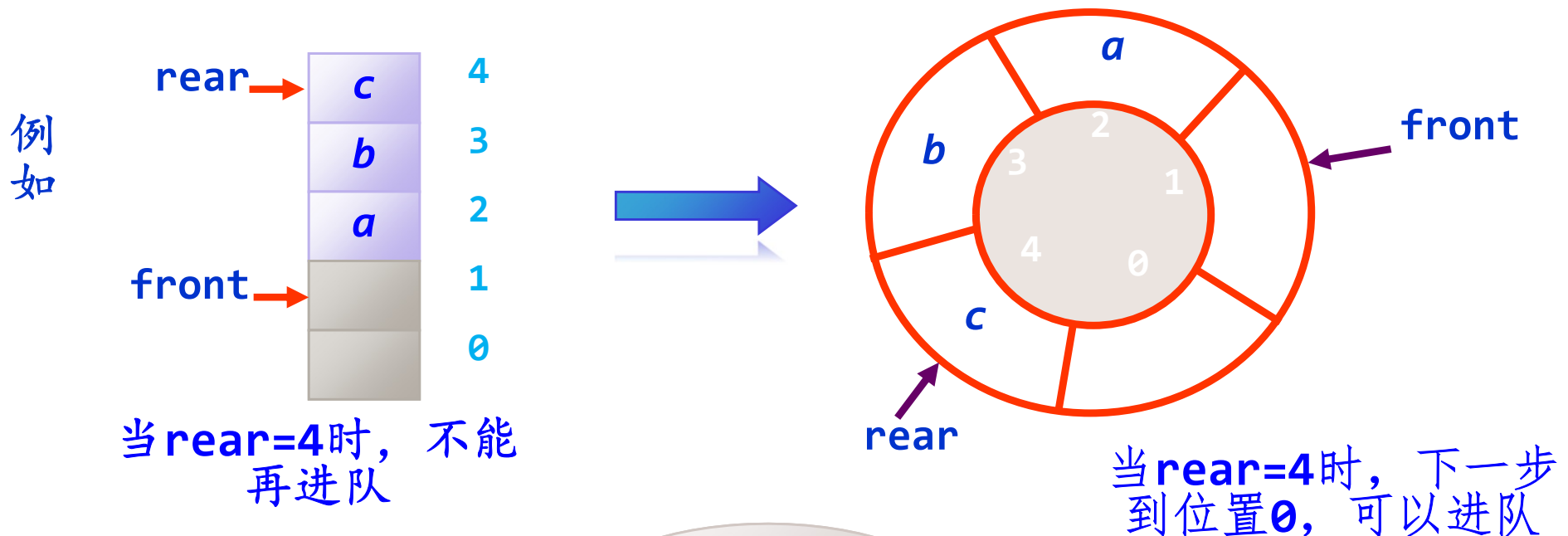


这是因为采用 $\text{rear} == \text{MaxSize} - 1$ 作为队满条件的缺陷。当队满条件为真时，队中可能还有若干空位置。

这种溢出并不是真正的溢出，称为**假溢出**。

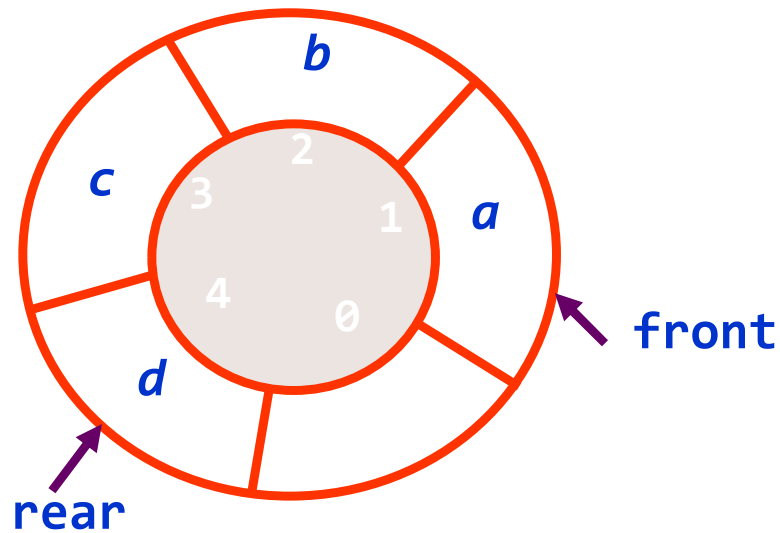
解决方案

把数组的前端和后端连接起来，形成一个环形的顺序表，即把存储队列元素的表从逻辑上看成一个环，称为**环形队列**或**循环队列**。



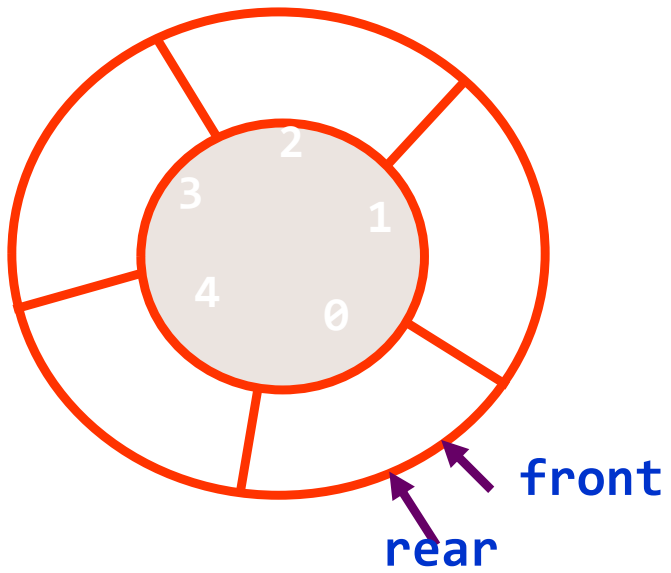


环形队列（循环队列）：

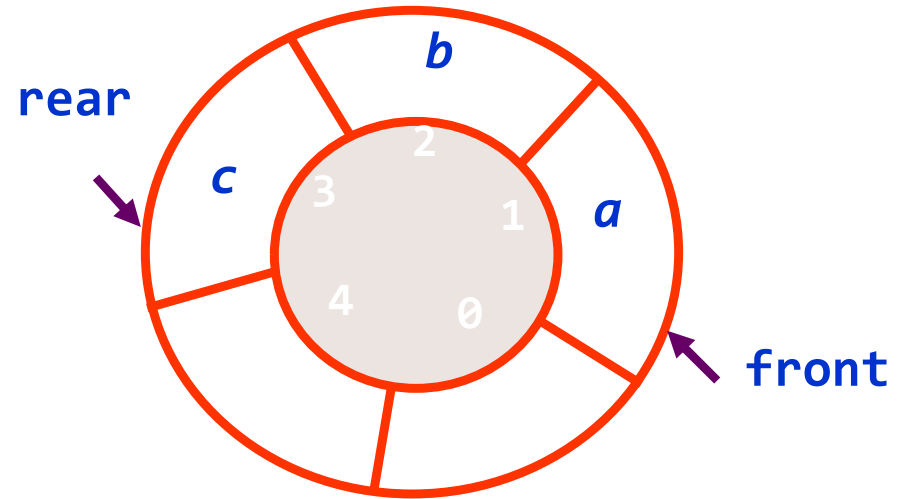


实际上内存地址一定是连续的，不可能是环形的，这里是通过逻辑方式实现环形队列，也就是将**rear++**和**front++**改为：

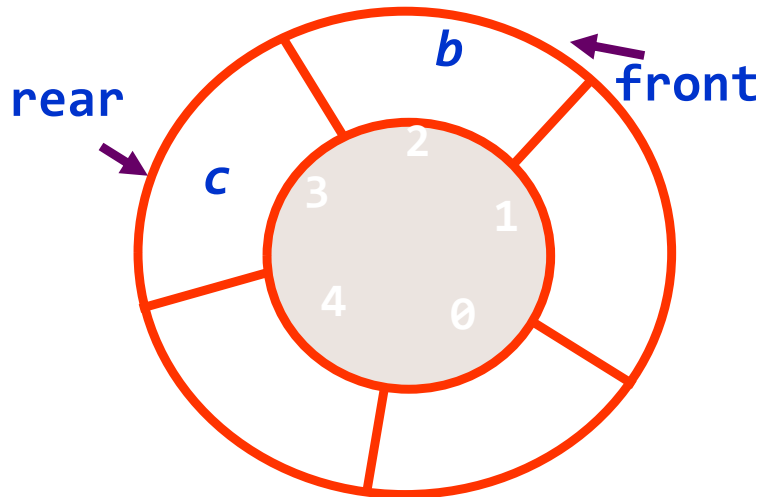
- **$\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$**
- **$\text{front} = (\text{front} + 1) \% \text{MaxSize}$**



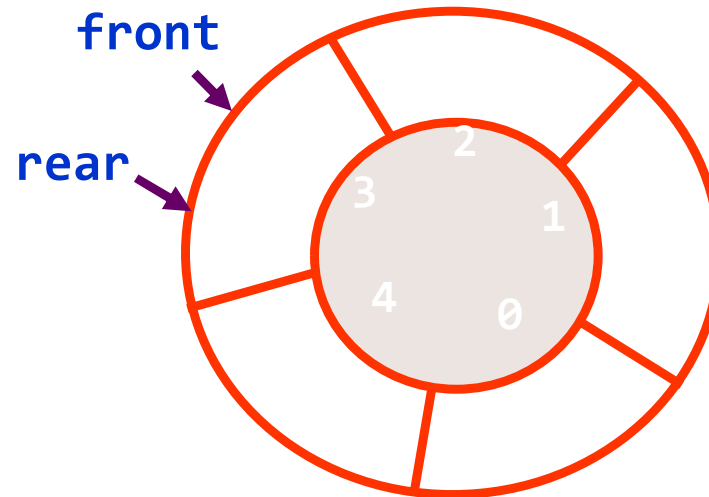
(a) 空队



(b) *a*、*b*、*c*进队



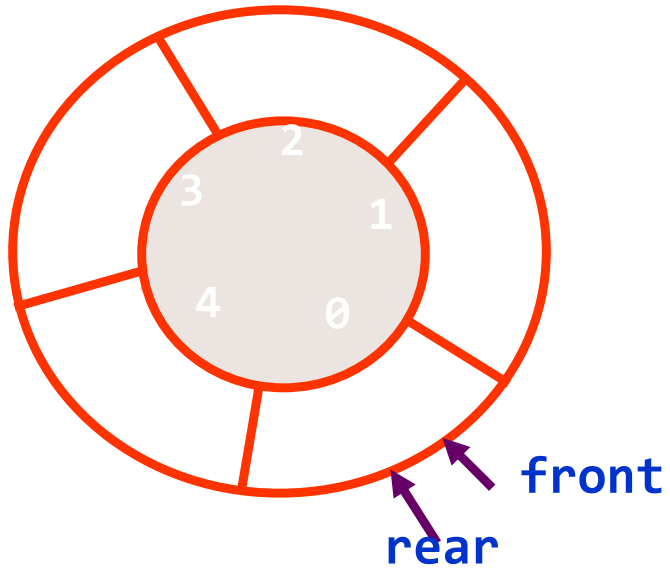
(c) 出队一次



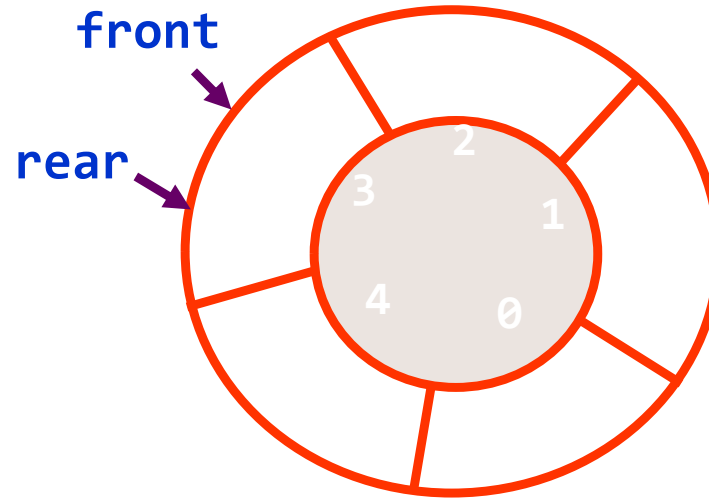
(d) 出队2次



现在约定 $\text{rear}=\text{front}$ 为队空，以下两种情况都满足该条件：



初始状态



进队的所有元素均出队

那么如何设置队满的条件呢？



让 $\text{rear} = \text{front}$ 为队空条件，并约定
 $(\text{rear} + 1) \% \text{MaxSize} = \text{front}$

为队满条件。

原因说明



进队一个元素时到
达队头，就认为队
满了。这样做会少
放一个元素，牺牲
一个元素没关系

- 通过相对值 $|\text{front} - \text{rear}|$ 标识不同的队列状态。
- 若 **data** 数组的容量为 m ，则队列有 $m+1$ 种状态，分别是队空、队中有 1 个元素、队中有 2 个元素、...、队中有 m 个元素（队满）。
- **front** 和 **rear** 的取值为 $0 \sim m-1$ ，这样 $|\text{front} - \text{rear}|$ 只有 m 个值。
- 队列 $m+1$ 种状态不能用 $|\text{front} - \text{rear}|$ 区分，有两种状态不能区分。



环形队列的4要素:

- 队空条件: $\text{front} = \text{rear}$
- 队满条件: $(\text{rear}+1)\% \text{MaxSize} = \text{front}$
- 进队 e 操作: $\text{rear}=(\text{rear}+1)\% \text{MaxSize}$; 将 e 放在 rear 处
- 出队操作: $\text{front}=(\text{front}+1)\% \text{MaxSize}$; 取出 front 处元素 e ;

在环形队列中, 实现队列的基本运算算法与非环形队列类似, 只是改为上述4要素即可。



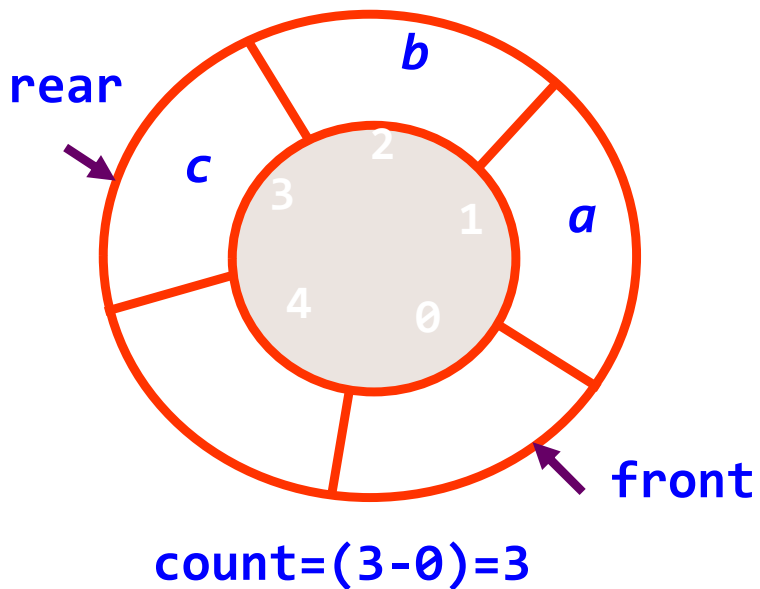
【例3.7】对于环形队列来说，如果知道队头指针和队列中元素个数，则可以计算出队尾指针。也就是说，可以用队列中元素个数代替队尾指针。

设计出这种环形队列的初始化、入队、出队和判空算法。



已知front、rear，求队中元素个数count = ?

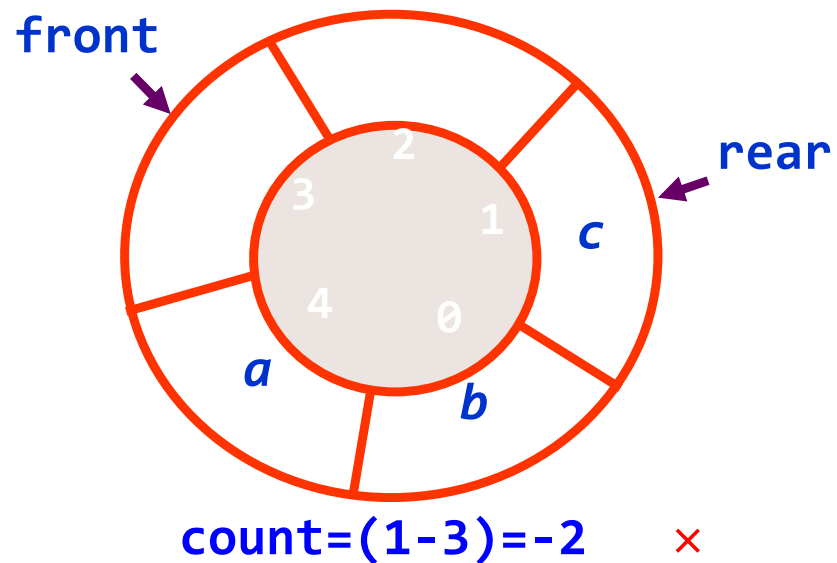
MaxSize = 5



count = rear - front ?

count = (3 - 0 + MaxSize) = 8 ×

count = (3 - 0 + MaxSize) % MaxSize = 3 ✓



count = (1 - 3 + MaxSize) = 3

count = (1 - 3 + MaxSize) % MaxSize = 3 ✓



已知front、rear，求队中元素个数count:

$$\text{count} = (\text{rear} - \text{front} + \text{MaxSize}) \% \text{MaxSize}$$

已知front、count，求rear:

$$\text{rear} = (\text{front} + \text{count}) \% \text{MaxSize}$$

已知rear、count，求front:

$$\text{front} = (\text{rear} - \text{count} + \text{MaxSize}) \% \text{MaxSize}$$



解

依题意设计的环形队列类型如下：

```
typedef struct
{   ElemType data[MaxSize];
    int front;           //队头指针
    int count;           //队列中元素个数
}   QuType;
```



该环形队列的4要素:

由front和count求出

- 队空条件: $\text{count} = 0$
- 队满条件: $\text{count} = \text{MaxSize}$
- 进队 e 操作: $\text{rear} = (\text{rear} + 1) \% \text{MaxSize}$; 将 e 放在 rear 处
- 出队操作: $\text{front} = (\text{front} + 1) \% \text{MaxSize}$; 取出 front 处元素 e ;

注意: 这样的环形队列中最多可放置 MaxSize 个元素。



对应的算法如下：

```
void InitQueue(QuType *&qu) //初始化队运算算法
{
    qu=(QuType *)malloc(sizeof(QuType));
    qu->front=0;
    qu->count=0;
}
```



```
bool EnQueue(QuType *&qu, ElemType x)           //进队运算算法
{  int rear;                                     //临时队尾指针
    if (qu->count==MaxSize)                       //队满上溢出
        return false;  它是一个局部变量，队列qu中
    else                                           不保存该值
    {  rear=(qu->front+qu->count)%MaxSize;         //求队尾位置
        rear=(rear+1)%MaxSize;                   //队尾循环增1
        qu->data[rear]=x;
        qu->count++;                               //元素个数增1
        return true;
    }
}
```



```
bool DeQueue(QuType *&qu, ElemType &x)           //出队运算算法
{  if (qu->count==0)                               //队空下溢出
    return false;
    else
    {  qu->front=(qu->front+1)%MaxSize;             //队头循环增1
      x=qu->data[qu->front];
      qu->count--;                                  //元素个数减1
      return true;
    }
}
```

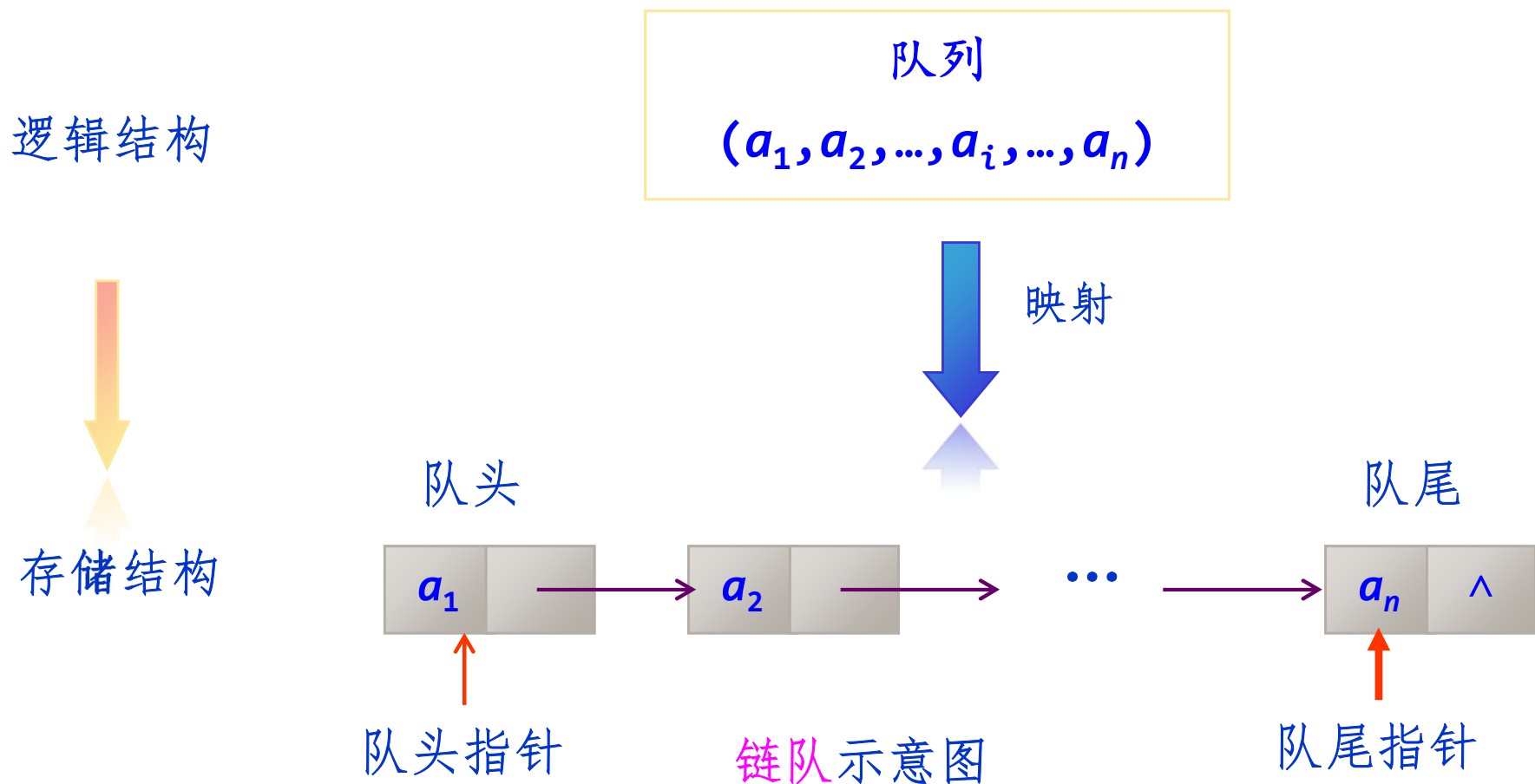


```
bool QueueEmpty(QuType *qu)    //判队空运算算法
{
    return(qu->count==0);
}
```



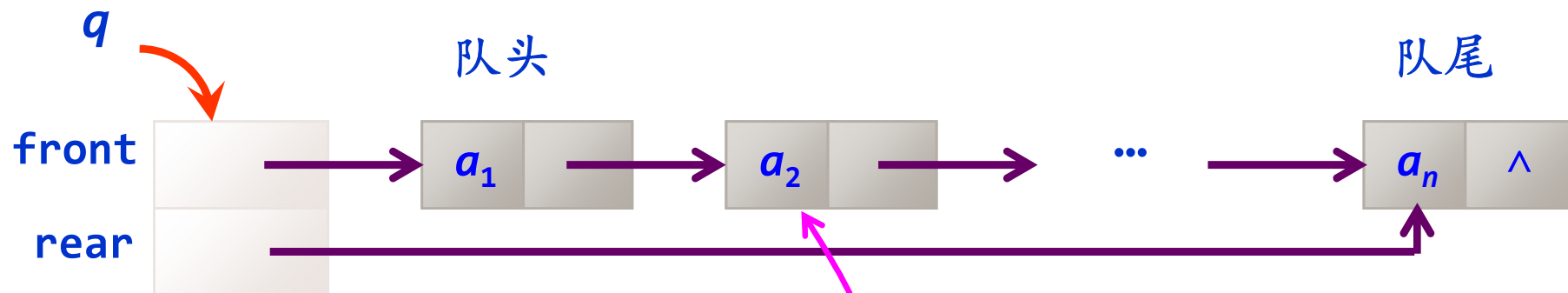
3.2.3 队列的链式存储结构及其基本运算的实现

采用链表存储的队列称为**链队**，这里采用不带头结点的单链表实现。





通常将队头和队尾两个指针合起来：



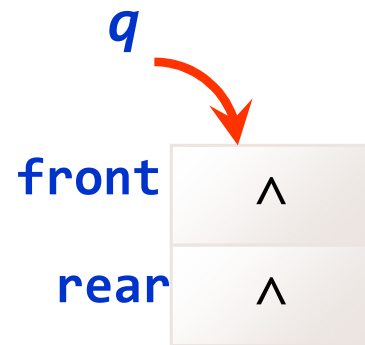
链队组成：

- (1) 存储队列元素的单链表结点
- (2) 指向队头和队尾指针的链队结点

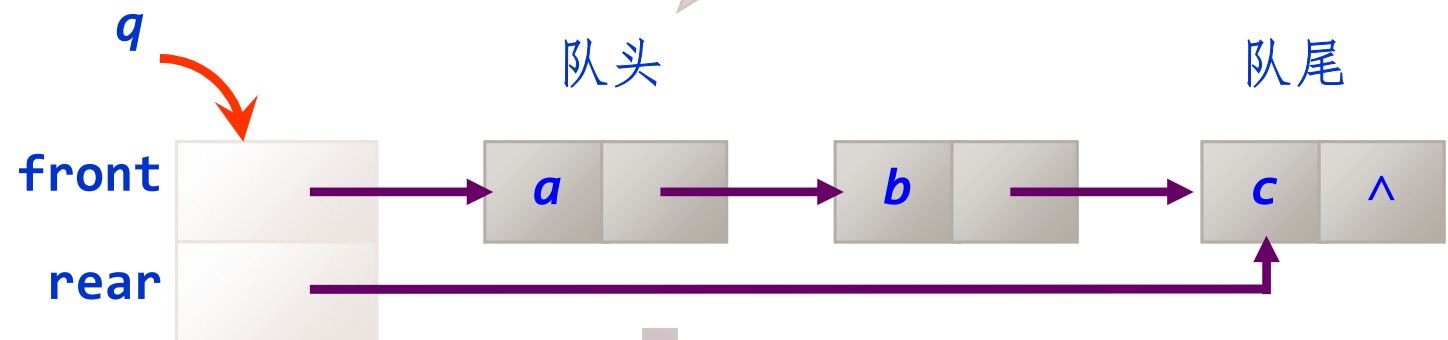


链队的进队和出队操作演示

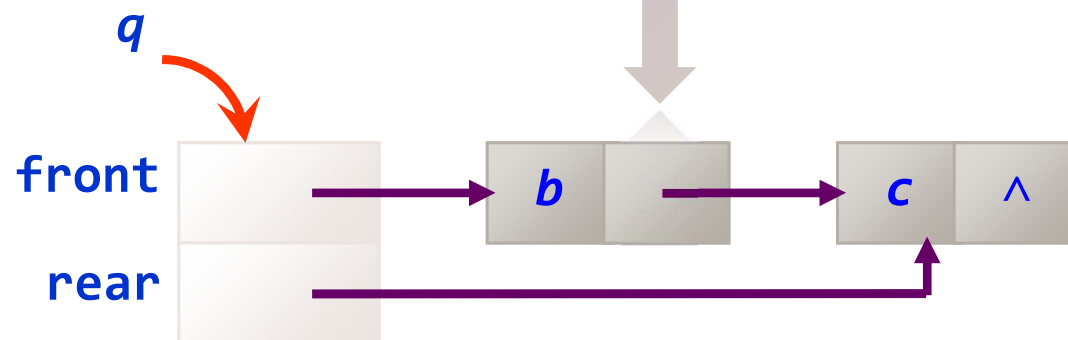
(a) 空队



(b) *a*、*b*、*c*进队



(c) 出队一次



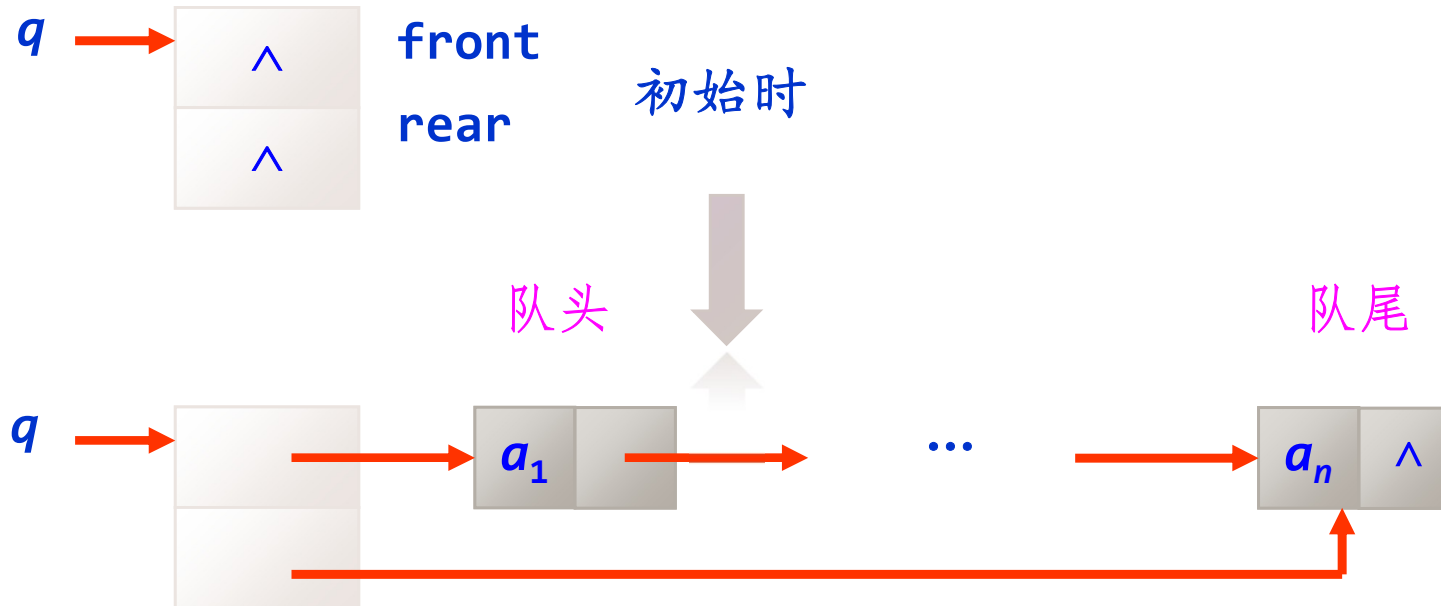


单链表中数据结点类型DataNode声明如下：

```
typedef struct qnode
{   ElemType data;      //数据元素
    struct qnode *next;
}   DataNode;
```

链队结点类型LinkQuNode声明如下：

```
typedef struct
{   DataNode *front;    //指向单链表队头结点
    DataNode *rear;     //指向单链表队尾结点
}   LinkQuNode;
```



链队的4要素:

- 队空条件: **front=rear=NULL**
- 队满条件: 不考虑
- 进队 e 操作: 将包含 e 的结点插入到单链表表尾
- 出队操作: 删除单链表首数据结点

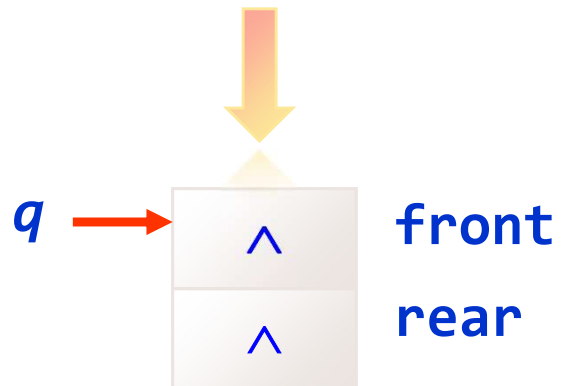


在链队存储中，队列的基本运算算法。

(1) 初始化队列InitQueue(q)

构造一个空队列，即只创建一个链队头结点，其**front**和**rear**域均置为**NULL**，不创建数据元素结点。

```
void InitQueue(LinkQuNode *&q)
{
    q=(LinkQuNode *)malloc(sizeof(LinkQuNode));
    q->front=q->rear=NULL;
}
```



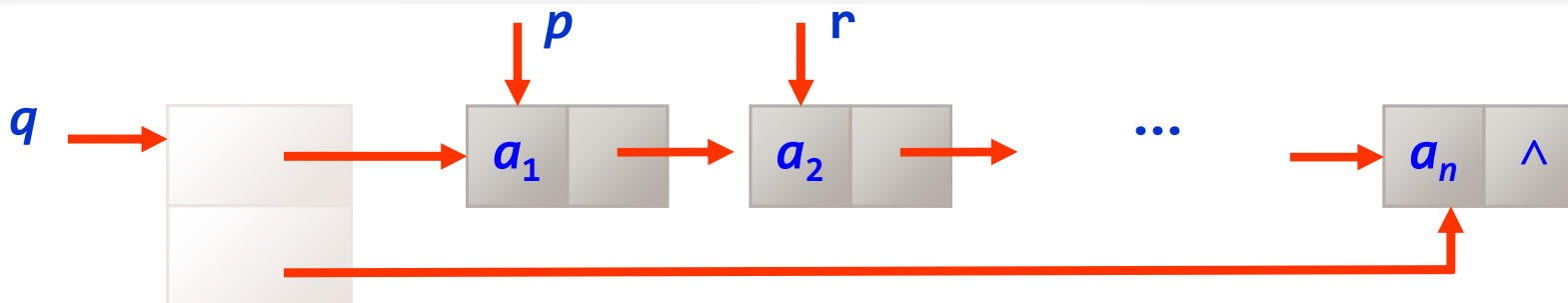
链队结点



(2) 销毁队列DestroyQueue(q)

释放队列占用的存储空间，包括链队头结点和所有数据结点的存储空间。

```
void DestroyQueue(LinkQuNode *&q)
{   DataNode *p=q->front, *r;           //p指向队头数据结点
    if (p!=NULL)                          //释放数据结点占用空间
    {   r=p->next;
        while (r!=NULL)
        {   free(p);
            p=r; r=p->next;
        }
    }
    free(p); free(q);                     //释放链队结点占用空间
}
```





(3) 判断队列是否为空QueueEmpty(q)

若链队结点的rear域值为NULL，表示队列为空，返回true；否则返回false。

```
bool QueueEmpty(LinkQuNode *q)
{
    return(q->rear==NULL);
}
```



空链队



(4) 进队enQueue(q, e)

考虑情况：

- 原队列为空
- 原队列非空



```
void enqueue(LinkQuNode *&q, ElemType e)
```

```
{ DataNode *p;
```

```
  p=(DataNode *)malloc(sizeof(DataNode));
```

```
  p->data=e;
```

```
  p->next=NULL;
```

```
  if (q->rear==NULL)    //若链队为空，新结点是队首结点又是队尾结点
```

```
      q->front=q->rear=p;
```

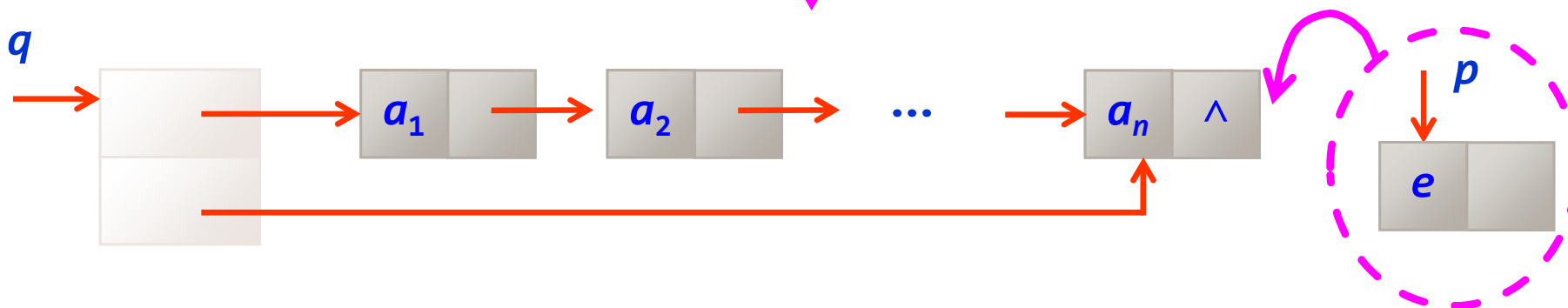
```
  else
```

```
  { q->rear->next=p; //将p结点链到队尾，并将rear指向它
```

```
    q->rear=p;
```

```
  }
```

```
}
```





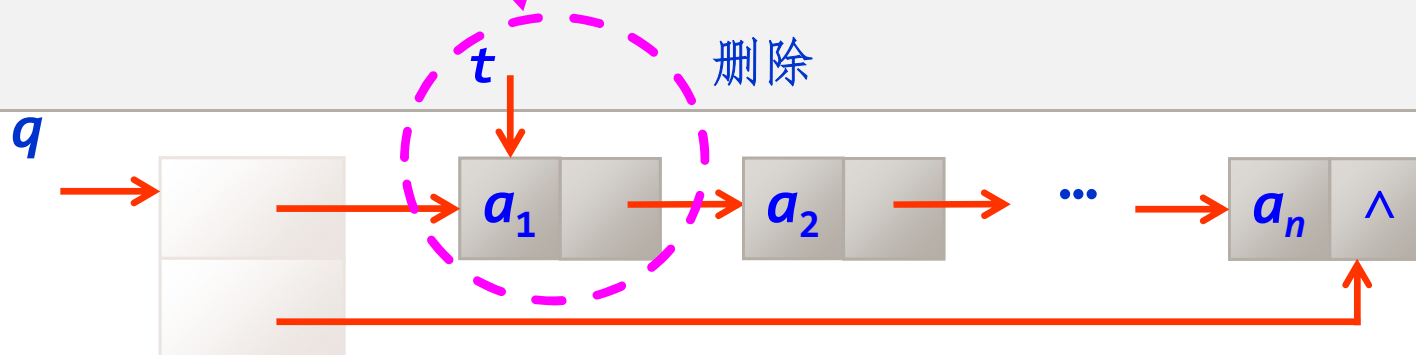
(5) 出队deQueue(q, e)

考虑情况：

- 原队列为空
- 原队列只有一个结点
- 其他情况

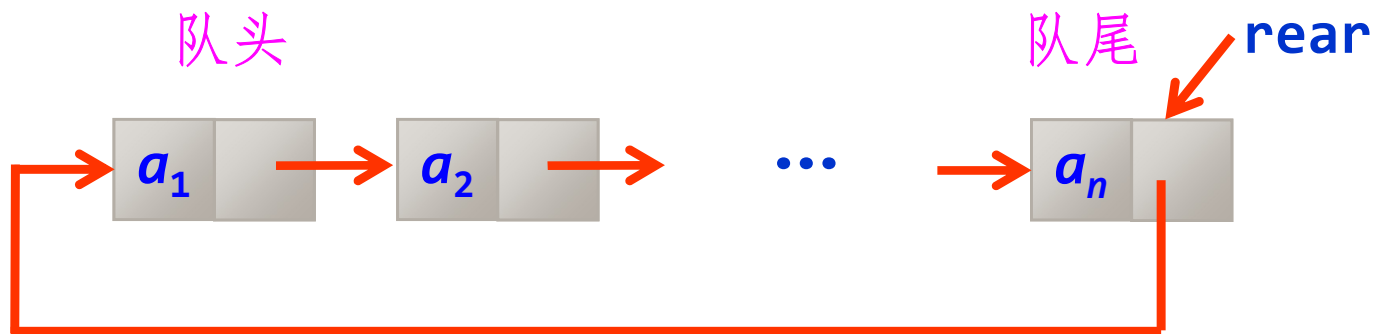


```
bool deQueue(LinkQuNode *&q, ElemType &e)
{
    DataNode *t;
    if (q->rear==NULL) return false; //队列为空
    t=q->front;                       //t指向第一个数据结点
    if (q->front==q->rear)             //队列中只有一个结点时
        q->front=q->rear=NULL;
    else                               //队列中有多个结点时
        q->front=q->front->next;
    e=t->data;
    free(t);
    return true;
}
```

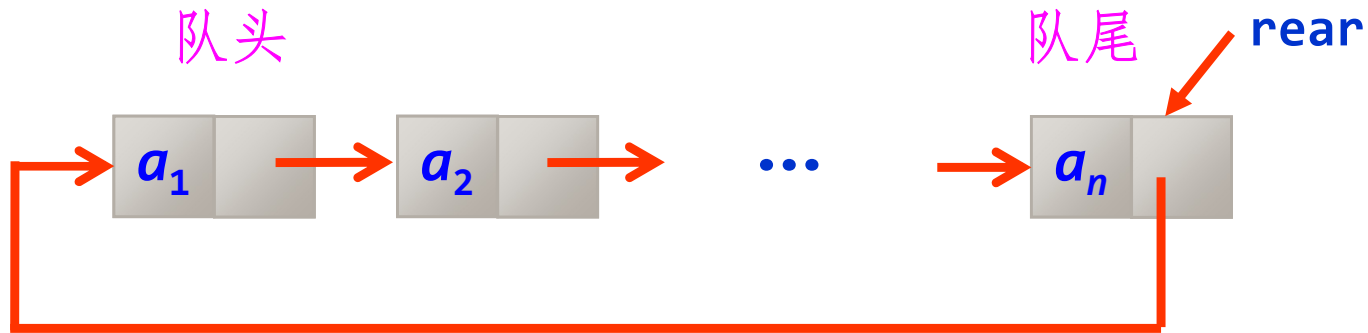




【例3.8】 采用一个不带头结点只有一个尾结点指针 rear 的循环单链表存储队列，设计队列的初始化、进队和出队等算法。



这样的链队通过尾结点指针 rear 唯一标识。



这样的链队通过尾结点指针rear唯一标识。

链队的4要素：

- 队空条件：rear=NULL
- 队满条件：不考虑
- 进队e操作：将包含e的结点插入到单链表表尾
- 出队操作：删除单链表首结点

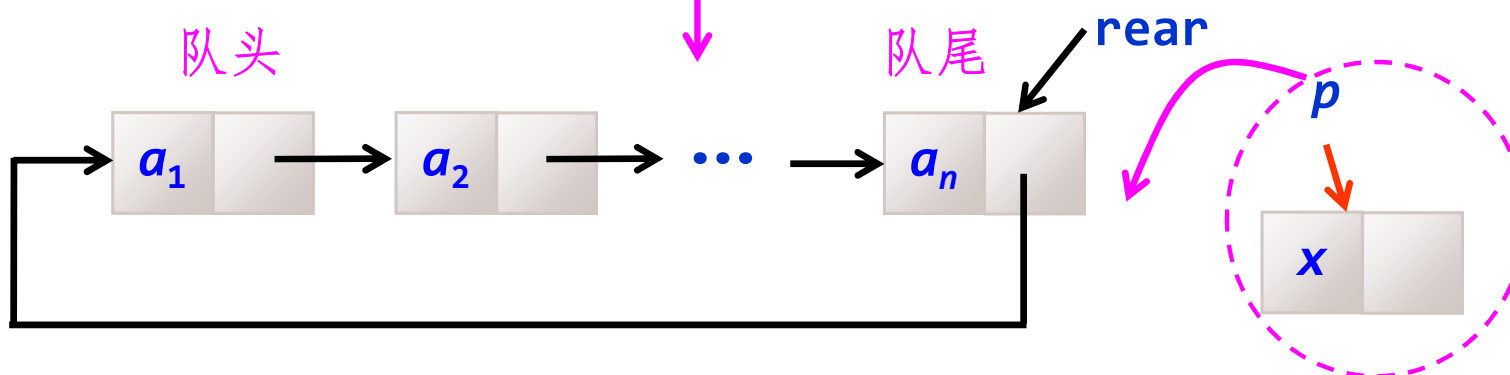


```
void InitQueue(LinkList *&rear)    //初始化队运算算法
{
    rear=NULL;
}

bool queueEmpty(LinkList *rear)    //判队空运算算法
{
    return(rear==NULL);
}
```

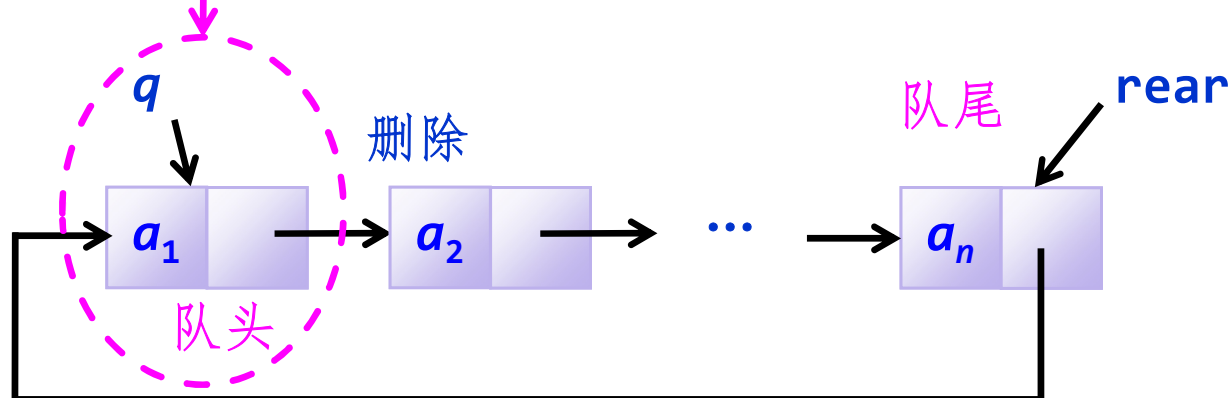


```
void enqueue(LinkList *&rear, ElemType x) //进队运算算法
{
    LinkList *p;
    p=(LinkList *)malloc(sizeof(LinkList)); //创建新结点
    p->data=x;
    if (rear==NULL) //原链队为空
    {
        p->next=p; //构成循环链表
        rear=p;
    }
    else
    {
        p->next=rear->next; //将p结点插入rear结点之后
        rear->next=p; //让rear指向新插入的结点
        rear=p;
    }
}
```





```
bool deQueue(LinkList *&rear, ElemType &x) //出队运算算法
{
    LinkList *q;
    if (rear==NULL) return false;           //队空
    else if (rear->next==rear)              //原队只有一个结点
    {
        x=rear->data;
        free(rear);
        rear=NULL;
    }
    else                                     //原队有两个或以上的结点
    {
        q=rear->next;
        x=q->data;
        rear->next=q->next;
        free(q);
    }
    return true;
}
```





3.2.4 队列的应用

1. 求解报数问题

问题描述

设有 n 个人站成一排，从左向右的编号分别为 $1 \sim n$ ，现在从左往右报数“1, 2, 1, 2, ...”，数到“1”的人出列，数到“2”的立即站到队伍的最右端。报数过程反复进行，直到 n 个人都出列为止。要求给出他们的出列顺序。

例如，当 $n=8$ 时，初始序列为

1 2 3 4 5 6 7 8

则出列顺序为

1 3 5 7 2 6 4 8



数据组织

用一个队列解决出队问题，由于这里不需要使用已经出队后的元素，所以采用环形队列。

- 环形队列中出队的元素可能被覆盖。
- 非环形队列中出队的元素不可能被覆盖。

- 如果利用队列求解时，希望结束后再利用所有出队的元素进一步查找结果，使用非环形队列，如迷宫问题。
- 否则，使用环形队列。



● 运算算法设计

算法思想：先将 n 个人的编号进队，然后反复执行如下操作，直到队列为空：

- ① 出队一个的元素，输出其编号—报数为1的人出列。
- ② 若队列不空，则再出队一个元素，并将刚出队的元素进队—报数为2的人站到队伍的最右端即队尾。



```
void number(int n)
{  int i;  ElemType e;
   SqQueue *q;                               //环形队列指针q
   InitQueue(q);                             //初始化队列q
   for (i=1;i<=n;i++)                        //构建初始序列
       enqueue(q,i);
   printf("报数出列顺序:");
   while (!QueueEmpty(q))
   {  dequeue(q,e);                          //队列不空循环
      printf("%d ",e);                       //出队一个元素e
      if (!QueueEmpty(q))                   //输出元素编号
      {  dequeue(q,e);                       //队列不空
         enqueue(q,e);                       //出队一个元素e
                                             //将刚出列的元素进队
      }
   }
   printf("\n");
   DestroyQueue(q);                          //销毁队列q
}
```



● 设计求解程序

```
int main()
{  int i,n=8;
   printf("初始序列:");
   for (i=1;i<=n;i++)
       printf("%d ",i);
   printf("\n");

   number(n);

   return 1;
}
```



运行结果

初始序列: 1 2 3 4 5 6 7 8

报数出列顺序: 1 3 5 7 2 6 4 8

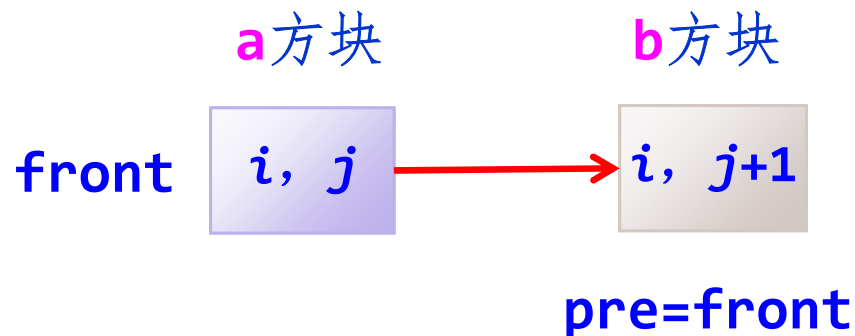


2. 求解迷宫问题

数据组织

使用一个队列`qu`记录试探的方块，该队列的结构如下：

```
typedef struct
{   int i,j;           //方块的位置
    int pre            //本路径中上一方块在队列中的下标
} Box;                      //方块类型
```





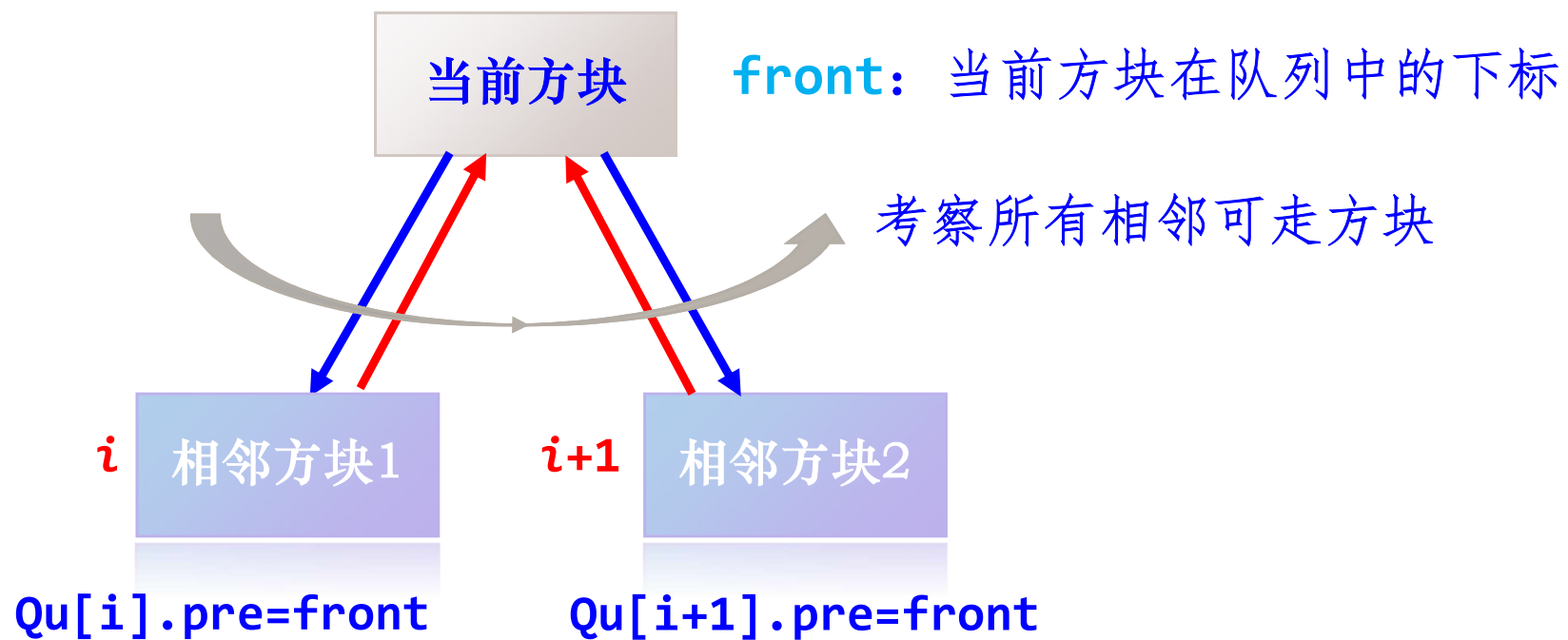
```
typedef struct
{
    Box data[MaxSize];
    int front, rear;    //队头指针和队尾指针
} QuType;             //定义顺序队类型
```

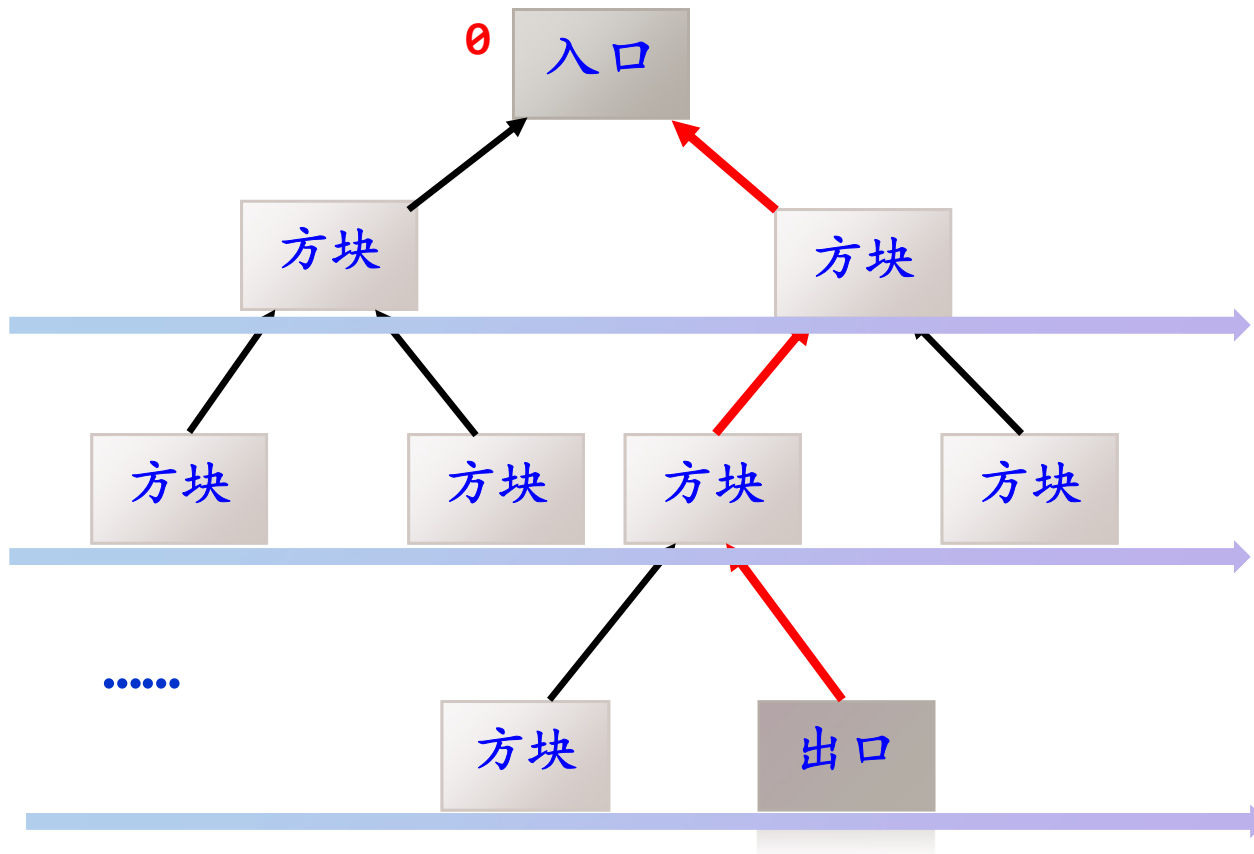
使用的队列qu不是环形队列（因为要利用出队的元素找路径），因此在出队时，不会将出队元素真正从队列中删除，因为要利用它输出路径。



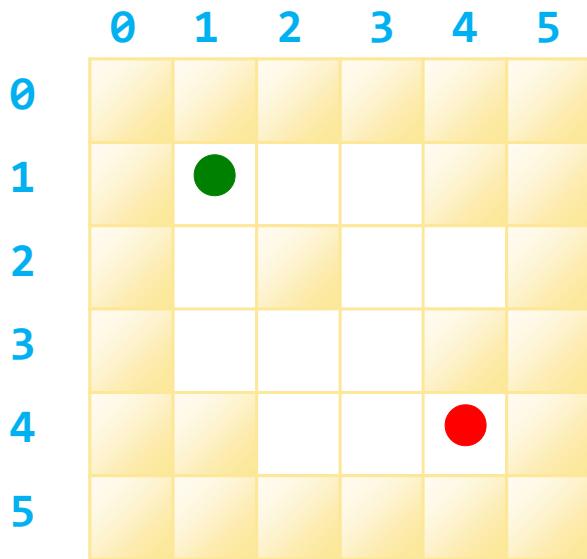
● 算法设计

首先将入口进队。出队一个方块，一次性考察如下：





- 所有搜索过的方块都在队列中。
- 最后通过队列找出从出口 $\Rightarrow$ 入口的一条迷宫路径。



入口

0:(1,1) -1

1:(1,2) 0

2:(2,1) 0

3:(1,3) 1

4:(3,1) 2

5:(2,3) 3

6:(3,2) 4

7:(2,4) 5 8:(3,3) 5 9:(4,2) 6

10:(4,3) 8

出口 → 11:(4,4) 10

```
int mg[M+2][N+2]=      //M=4,N=4
{ {1, 1, 1, 1, 1, 1},
  {1, 0, 0, 0, 1, 1},
  {1, 0, 1, 0, 0, 1},
  {1, 0, 0, 0, 1, 1},
  {1, 1, 0, 0, 0, 1},
  {1, 1, 1, 1, 1, 1} };
```

迷宫路径:

(4,4)

(4,3)

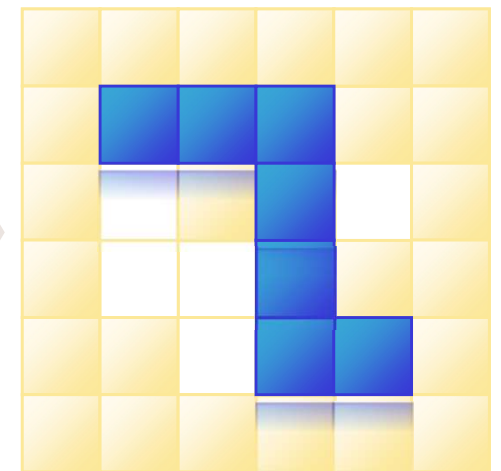
(3,3)

(2,3)

(1,3)

(1,2)

(1,1)





用队列求一条迷宫路径的算法: $(x_i, y_i) \Rightarrow (x_e, y_e)$

```
bool mgpath1(int xi, int yi, int xe, int ye)
//搜索路径为:  $(x_i, y_i) \Rightarrow (x_e, y_e)$ 
{   Box e;
    int i, j, di, i1, j1;
    QuType *qu;                //定义顺序队指针qu
    InitQueue(qu);             //初始化队列qu
    e.i=xi; e.j=yi; e.pre=-1;
    enqueue(qu,e);             //(xi, yi)进队
    mg[xi][yi]=-1;             //将其赋值-1,以避免回过来重复搜索
```



```
while (!QueueEmpty(qu))           //队不空循环
{   dequeue(qu,e);                 //出队方块e
    i=e.i;   j=e.j;
    if (i==xe && j==ye)             //找到了出口,输出路径
    {   dispapath(qu,qu->front);     //调用dispapath函数输出路径
        DestroyQueue(qu);          //销毁队列
        return true;               //找到一条路径时返回真
    }
```



```
for (di=0;di<4;di++)           //循环扫描每个方位
{
    switch(di)
    {
        case 0:i1=i-1; j1=j;    break;
        case 1:i1=i;   j1=j+1; break;
        case 2:i1=i+1; j1=j;    break;
        case 3:i1=i;   j1=j-1; break;
    }
    if (mg[i1][j1]==0)
    {
        e.i=i1;  e.j=j1;
        e.pre=qu->front;
        enqueue(qu,e);  //(i1, j1)方块进队
        mg[i1][j1]=-1;  //将其赋值-1
    }
}
DestroyQueue(qu);              //销毁队列
return false;
}
```

把每个可走的方块插入队列中



对于图3.11的迷宫，求解 $(1,1) \rightarrow (8,8)$ 时队列qu中结果如下：

下标	i	j	pre
0	1	1	-1
1	1	2	0
2	2	1	0
3	2	2	1
4	3	1	2
5	3	2	3
6	4	1	4
7	3	3	5
8	5	1	6
9	3	4	7
10	5	2	8
11	6	1	8
12	2	4	9
13	5	3	10
14	7	1	11

下标	i	j	pre
15	1	4	12
16	2	5	12
17	6	3	13
18	1	5	15
19	2	6	16
20	6	4	17
21	1	6	18
22	6	5	20
23	5	5	22
24	7	5	22
25	4	5	23
26	5	6	23
27	8	5	24
28	4	6	25
29	5	7	26

下标	i	j	pre
30	8	6	27
31	8	4	27
32	4	7	28
33	5	8	29
34	6	7	29
35	8	7	30
36	8	3	31
37	3	7	32
38	4	8	32
39	6	8	33
40	8	8	35

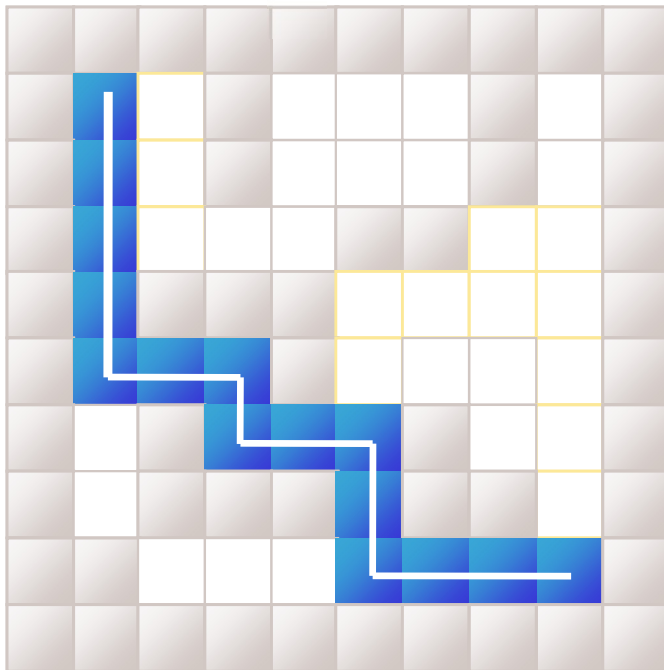


运行结果

对于图3.11的迷宫，求解结果如下：

迷宫路径如下：

(1, 1)	(2, 1)	(3, 1)	(4, 1)	(5, 1)
(5, 2)	(5, 3)	(6, 3)	(6, 4)	(6, 5)
(7, 5)	(8, 5)	(8, 6)	(8, 7)	(8, 8)



显然，这个解是最优解，即是最短路径。为什么？



3.2.5 双端队列

所谓**双端队列**是指两端都可以进行进队和出队操作的队列，将队列的两端分别称为前端和后端，两端都可以入队和出队。

其元素的逻辑结构仍是线性结构。





例如，有 a 、 b 、 c 、 d 元素进队，能否产生 $dcab$ 的出队序列呢？答案是肯定的。

前端

后端



出队序列

进队序列 a b c d



- **输出受限的双端队列**，即一个端点允许插入和删除，另一个端点只允许插入的双端队列。





【例3.10】某队列允许在两端进行入队操作，但仅允许在一端进行出队操作，若 a 、 b 、 c 、 d 、 e 元素进队，则不可能得到的顺序是（ ）。

A. $bacde$ B. $dbace$ C. $dbcae$ D. $ecbad$

说明：本题为2010年全国考研题。

答案为C。

前端

后端



出队序列

进队序列 a b c d e

选项C操作： a 后端进， b 前端进，因 d 未出，此时只能进队， c 怎么进都不可能在 b 、 a 之间。



- **输入受限的双端队列**，即一个端点允许插入和删除，另一个端点只允许删除的双端队列。





第3章 小结

1

栈

① 先进后出表。1, 2, 3, ..., n通过一个栈的出栈序列个数?

$$\frac{1}{n+1} C_{2n}^n = \frac{1}{n+1} \times \frac{(2n)!}{n! \times n!} \quad \leftarrow \text{第 } n \text{ 个 Catalan 数}$$

当 $n=3$, 结果为5。



② 一个大小为 n 的顺序栈，最多只能进行 n 次进栈操作吗？

错误：可以进行任意多次进栈操作。

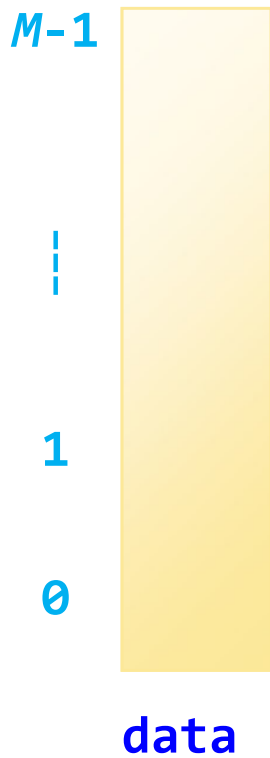
但最多只能进行连续 n 次进栈操作。



③ 顺序栈只能将栈底设置在data[0]端吗?

可以将栈底设置在任意一端，但不能设置在中间。

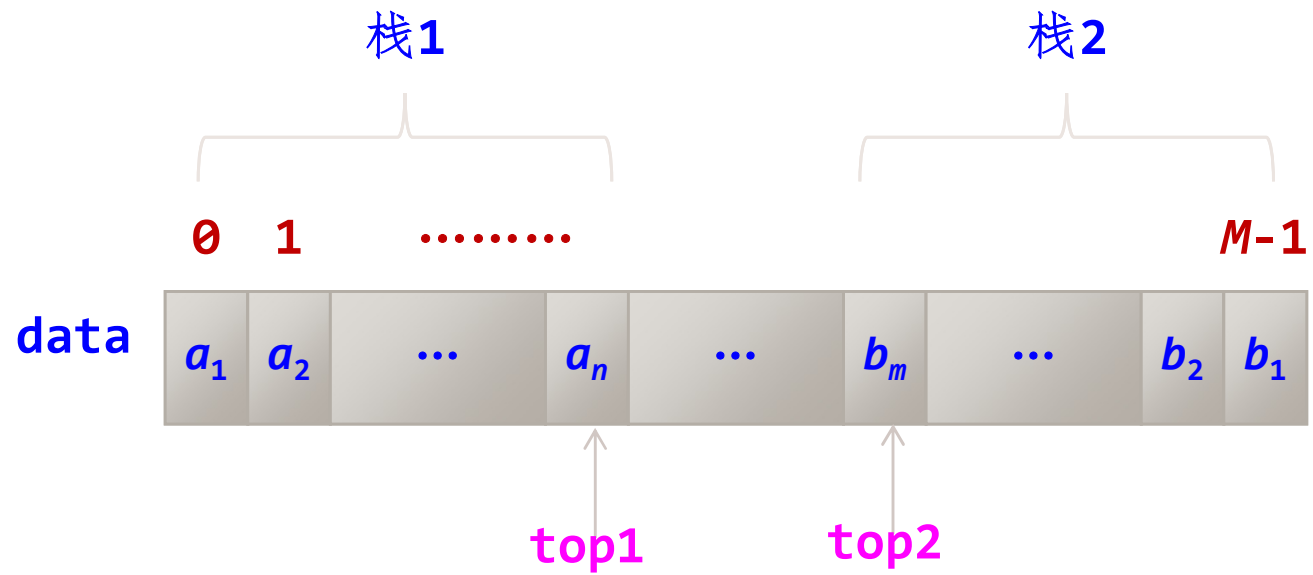
将栈底设置在data[M-1]端的设计：



- 初始化: $top=M$
- 栈空: $top==M$
- 栈满: $top==0$ (最小下标)
- 元素 e 进栈: $top--$; $data[top]=e$
- 出栈: $e=data[top]$; $top++$



④ 共享栈





2 队 列

① 先进先出表。1, 2, 3, ..., n 通过一个队列的出队序列个数?

只有一个: 即1, 2, 3, ..., n



② 环形队列解决了假溢出问题，任何情况下都使用环形队列吗？

- 采用环形队列时，进队的元素可能被覆盖。
- 如果需要用队列中全部进队的元素进一步求解问题，应该采用非环形队列。如用队列求解迷宫路径！



③ 如果需要多个队列，可以像共享栈一样设置共享队列吗？如果需要10个队列，如何设计？

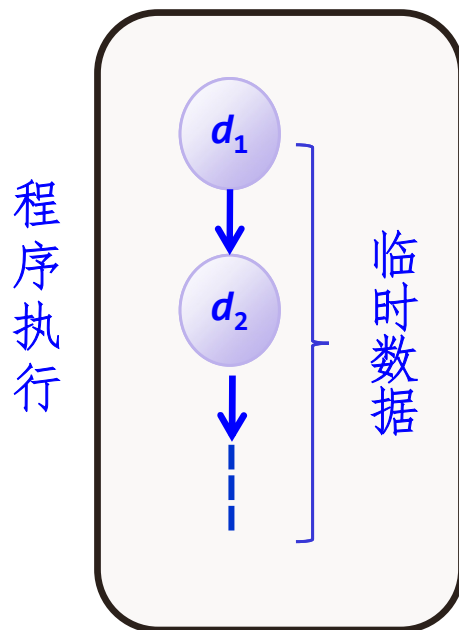
不能。因为栈是向一端伸展的，而队列不是。为了节省空间，应该采用链队。

如果需要使用10个队列，可以设置10个链队：

- 队头指针：front[10]
- 队尾指针：rear[10]

3

栈和队列的应用



保存 d_1 、 d_2 、...

- 先产生的数据后处理—栈（后进先出表）
- 先产生的数据先处理—队列（先进先出表）



实验练习

以下两类2选1即可:

- **类别1**: Leetcode, 题号: 20、150、225、227、232、234、387、862、1047、LCR 027

- **类别2**:
 - ✓ P115, 练习题3, 题号: 11、13;
 - ✓ P117, 上机实验3, 题号: 实验题1、实验题4;
 - ✓ P118, 设计性实验, 题号: 实验题5、实验题7;
 - ✓ P119, 综合性实验, 题号: 实验题8;
 - ✓ P120, LeetCode在线编程3, 题号: 1、4、12